

IN-THE-FLOW AGENTIC SYSTEM OPTIMIZATION FOR EFFECTIVE PLANNING AND TOOL USE

Zhuofeng Li^{*1,2}, Haoxiang Zhang^{*1,3}, Seungju Han¹, Sheng Liu¹, Jianwen Xie⁴,
Yu Zhang², Yejin Choi¹, James Zou^{†1}, Pan Lu^{†1}

¹Stanford University, ²Texas A&M University, ³UC San Diego, ⁴Lambda



Website: <https://agentflow.stanford.edu>



Code



Model



Demo



Visualize

ABSTRACT

Outcome-driven reinforcement learning has advanced reasoning in large language models (LLMs), but prevailing tool-augmented approaches train a single, monolithic policy that interleaves thoughts and tool calls under full context; this scales poorly with long horizons and diverse tools and generalizes weakly to new scenarios. Agentic systems offer a promising alternative by decomposing work across specialized modules, yet most remain training-free or rely on offline training decoupled from the live dynamics of multi-turn interaction. We introduce AGENTFLOW, a trainable, *in-the-flow* agentic framework that coordinates four modules (planner, executor, verifier, generator) through an evolving memory and directly optimizes its planner inside the multi-turn loop. To train on-policy in live environments, we propose *Flow-based Group Refined Policy Optimization* (Flow-GRPO), which tackles long-horizon, sparse-reward credit assignment by converting multi-turn optimization into a sequence of tractable single-turn policy updates. It broadcasts a single, verifiable trajectory-level outcome to every turn to align local planner decisions with global success and stabilizes learning with group-normalized advantages. Across ten benchmarks, AGENTFLOW with a 7B-scale backbone outperforms top-performing baselines with average accuracy gains of 14.9% on search, 14.0% on agentic, 14.5% on mathematical, and 4.1% on scientific tasks, even surpassing larger proprietary models like GPT-4o. Further analyses confirm the benefits of in-the-flow optimization, showing improved planning, enhanced tool-calling reliability, and positive scaling with model size and reasoning turns.

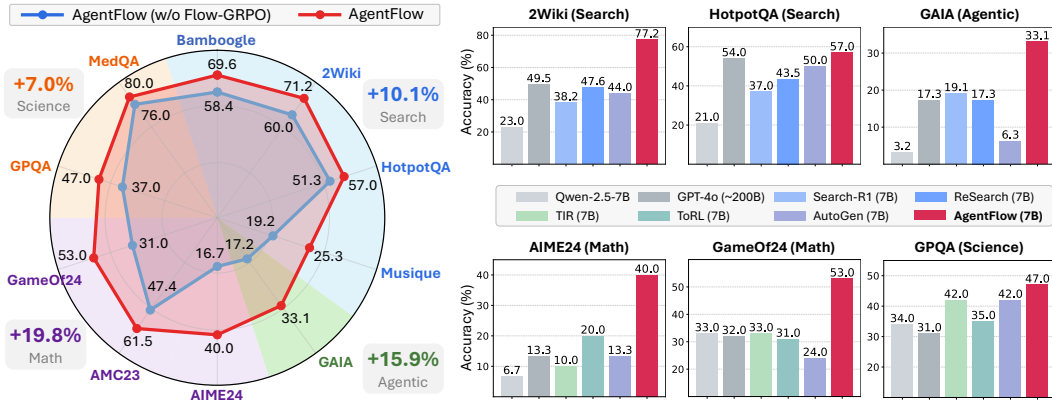


Figure 1: **Left:** Performance of AGENTFLOW with a 7B-scale backbone before and after Flow-GRPO tuning across ten diverse reasoning benchmarks. Flow-GRPO substantially improves performance by enhancing planning quality and tool-calling reliability. **Right:** AGENTFLOW achieves consistent gains over top baselines, including base LLMs, tool-integrated RL models, and training-free agentic systems. All 7B results use Qwen2.5-7B-Base/Instruct as the backbone and tools.

^{*}Equal contribution. [†]Co-senior authors. Work was partially done while ZL and HZ were visiting Stanford.

1 INTRODUCTION

Recent advances in large language models (LLMs) have unlocked remarkable reasoning capabilities, largely driven by reinforcement learning (RL) from outcome-based feedback. By fine-tuning models to maximize verifiable rewards, LLMs like DeepSeek-R1 (Guo et al., 2025) and SimpleRL (Zeng et al., 2025b) have demonstrated sophisticated behaviors in self-correction and multi-step deduction.

A complementary line of work augments LLMs with external tools (e.g., web search, code execution) for knowledge retrieval and precise computation. Tool-integrated reasoning (TIR) extends reinforcement learning with verifiable rewards to learn *when* and *how* to call tools by interleaving reasoning (e.g., `<think>`) with tool invocations (e.g., `<tool_call>`) under full context (Jin et al., 2025; Song et al., 2025; Chen et al., 2025; Feng et al., 2025). Early systems supported only a single tool type, whereas recent work enables multi-tool settings by encoding tool metadata into prompts (Dong et al., 2025; Qian et al., 2025a; Zhang et al., 2025). However, these methods still train a *single*, monolithic policy under multi-turn full-context reasoning, which introduces scaling challenges: (i) *training* becomes increasingly unstable as horizons lengthen, tool diversity grows, and environments shift with tool feedback (Wang et al., 2025c; Mai et al., 2025; Moonshot AI, 2025; Xue et al., 2025); and (ii) *inference*-time generalization remains brittle to unseen tasks or tools (Dong et al., 2025; Hu et al., 2025b).

Agentic systems (Wu et al., 2024; Hong et al., 2024; Hu et al., 2025b) offer a promising alternative to monolithic tool-integrated reasoning models. They consist of multiple modules—often distinct LLMs with prescribed roles (e.g., planner, critic) or specialized components with dedicated tools and capabilities (e.g., executor, coder)—that coordinate via shared memory and inter-module communication. By decomposing problems into sub-goals and iterating over multiple turns, these systems can tackle tasks that demand diverse tools, long horizons, or multi-stage reasoning. However, achieving robust coordination in such systems ultimately requires *training*, since handcrafted logic or static prompting cannot reliably capture when and how modules should collaborate, adapt to evolving tool outputs, or recover from early mistakes. At the same time, they introduce new *training* challenges: modules coordinate sequentially, outcome feedback propagates through long reasoning chains, and state distributions shift with evolving tool outputs. As a result, most systems remain *training-free*, relying on handcrafted logic or prompting heuristics. While some employ supervised fine-tuning or preference optimization for key modules (Motwani et al., 2024; Park et al., 2025), these off-policy approaches are decoupled from live dynamics and learn poorly from downstream successes or failures. Thus, agentic systems struggle with sparse rewards, brittle adaptation, and inefficient orchestration in dynamic environments.

To address the central challenge of learning long-horizon reasoning with sparse rewards in tool-integrated agentic systems, we introduce AGENTFLOW, a *trainable* framework for effective planning and tool use (Figure 2). AGENTFLOW comprises four specialized modules—planner, executor, verifier, and generator—that interact iteratively over multiple turns via a shared evolving memory and a toolset. The system operates *in the flow*, with each turn cycling through planning, execution, and verification. Unlike prior agentic systems, AGENTFLOW directly optimizes its planner on-policy, *inside* the live multi-turn loop, allowing it to dynamically adapt to trajectories shaped by tool calls, verifier signals, and memory updates. This evolving memory serves as a deterministic, structured record of the reasoning process, enabling transparent state tracking, controllable behavior, and bounded context growth.

To train the planner on-policy within this agentic system, we need to overcome the long-horizon credit assignment problem inherent to sparse, trajectory-level rewards. We introduce *Flow-based Group Refined Policy Optimization* (Flow-GRPO, Figure 4), an on-policy algorithm designed for this setting. Flow-GRPO operates on *in-the-flow* rollouts, which capture the full trajectory of states, actions, and tool events induced by the live system. Instead of attempting to assign credit with brittle, intermediate heuristics, we assign a single, verifiable final-outcome reward to the entire trajectory and *broadcast* it to every turn. This design effectively transforms the multi-turn reinforcement learning challenge into a series of single-turn updates: at each turn, the planner has access to the full memory context and receives a consistent reward signal aligned with global success. This approach, coupled with group-normalized advantages to stabilize training, enables robust credit assignment and allows the planner to learn effective long-horizon strategies from sparse feedback.

We evaluate AGENTFLOW on ten benchmarks across diverse reasoning domains, as results highlighted in Figure 1. In our main setting, all four modules use Qwen2.5-7B-Instruct (Yang et al.,

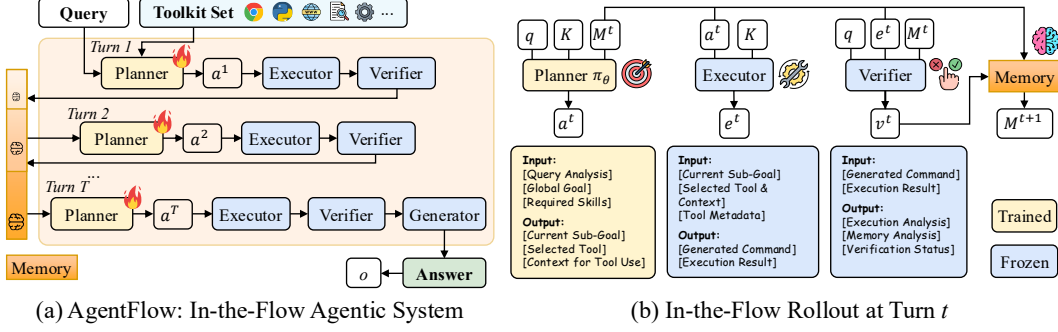


Figure 2: **(a)** Overview of AGENTFLOW, a trainable agentic system for in-the-flow planning and tool use. Four modules (planner, executor, verifier, generator) coordinate via a shared evolving memory M and toolset K , given a query q . The planner policy is optimized on-policy *inside* the system’s multi-turn loop to enable adaptive, long-horizon reasoning. **(b)** A single state transition, showing the action a^t , execution result e^t , and verifier signal v^t that update the memory from M^t to M^{t+1} .

2024a) as a backbone, with only the planner trained via Flow-GRPO. AGENTFLOW substantially outperforms top-performing specialized tool-integrated reasoning models and agentic systems, achieving average accuracy by 14.9% on knowledge-intensive search, 14.0% on broader agentic tasks, 14.5% on mathematical reasoning, and 4.1% on scientific reasoning (§4.2). Notably, our 7B-backbone system even surpasses the $\sim 200\text{B}$ -parameter GPT-4o (Hurst et al., 2024) across all domains. The trained planner learns to optimize planning, enhance tool-calling reliability, and discover effective solution pathways (§4.3). Further analyses confirm that our in-the-flow optimization with Flow-GRPO is crucial, far surpassing offline supervised tuning (§4.4). Moreover, our training approach proves highly efficient, leading to increased rewards and condensed responses compared to traditional tool-integrated RL methods (§4.5). Finally, we demonstrate that these benefits generalize, with consistent gains from scaling backbone size and turn budget (§4.6).

Our work makes three key contributions: (1) We present AGENTFLOW, a trainable *in-the-flow* agentic system that directly optimizes its planner *inside* the multi-turn loop. By coordinating specialized modules through an evolving memory, it enables adaptive long-horizon planning and robust tool orchestration. (2) We introduce *Flow-GRPO*, an on-policy, outcome-driven algorithm that *converts* multi-turn RL into a sequence of tractable *single-turn* policy updates by *broadcasting* a single, verifiable final-outcome reward to every turn. (3) Through comprehensive experiments on ten benchmarks, we show that AGENTFLOW with a 7B backbone outperforms specialized baselines and even larger proprietary models. Further analyses reveal improved planning, enhanced tool-calling reliability, and positive scaling with model size and turn budgets.

2 PRELIMINARY

Reinforcement learning for reasoning LLMs. Recent progress in reasoning LLMs has been significantly driven by reinforcement learning from outcome feedback, using a verifiable reward signal (Shao et al., 2024; Yu et al., 2025). This paradigm fine-tunes a language model to maximize an outcome-based reward while remaining close to a reference policy. Formally, the objective is to optimize a policy LLM π_θ to generate a response o for a given query q from dataset $\mathcal{D}$:

$$\max_{\pi_\theta} \mathbb{E}_{x \sim \mathcal{D}, o \sim \pi_\theta(\cdot|q)} [R(q, o)] - \beta \mathbb{D}_{\text{KL}}(\pi_\theta(o|q) \parallel \pi_{\text{ref}}(o|q)), \quad (1)$$

where $R(q, o)$ is the outcome-based reward, π_{ref} is a reference model to prevent policy collapse, and β controls KL regularization. Algorithms like Group Relative Policy Optimization (GRPO) (Shao et al., 2024) implement this by sampling groups of responses, normalizing advantages by their rewards, and updating the policy with a clipped objective to encourage high-reward outputs.

Tool-integrated reasoning models (LLM agents). LLMs can be augmented with external tools to access knowledge and perform precise computation under reinforcement learning with outcome-based reward. As shown in Figure 3(a), the LLM *interleaves* reasoning and tool calls, producing a chain of thought within `<think></think>` tokens followed by tool invocations (e.g., `<tool_call></tool_call>`). The resulting trajectory τ is a sequence of model generations and tool observations: $\tau = \{s^1, a^1, e^1, \dots, s^T, a^T\}$, where s^t denotes the context, a^t the generated action (thought + tool call), and e^t the tool’s execution result. The policy model π_θ is then trained to

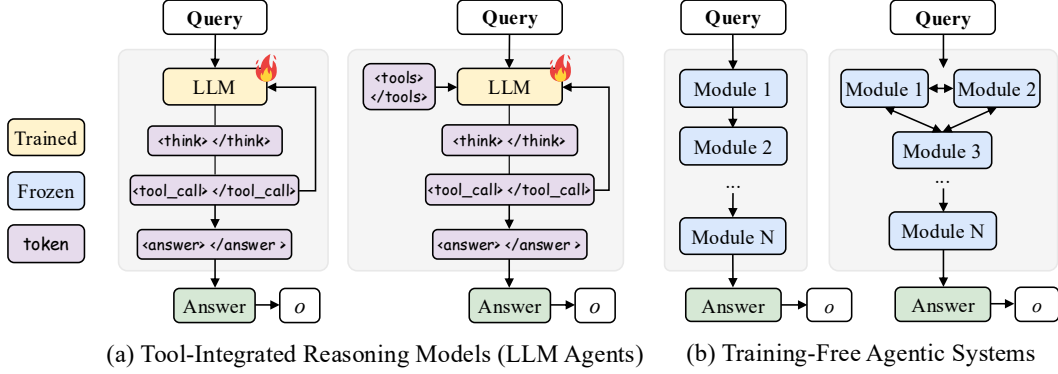


Figure 3: **Comparison of two paradigms of LLMs with tool use.** (a) Monolithic tool-integrated reasoning models train a single policy to interleave reasoning (e.g., `<think>`) and tool calls (e.g., `<tool_call>`) within a single, full-context trajectory. (b) Agentic systems decompose tasks across multiple specialized modules (e.g., planner, coder) that collaborate. These systems are typically training-free, orchestrated by handcrafted logic or prompting.

maximize a final outcome reward. Prior work has explored single- and multi-tool settings for search and code execution (Jin et al., 2025; Chen et al., 2025; Feng et al., 2025; Qian et al., 2025a).

Agentic systems with tool usage. An alternative approach is the use of agentic systems (Wu et al., 2024; Hong et al., 2024; Lu et al., 2025). As shown in Figure 3(b), these frameworks deploy multiple specialized modules—often distinct LLMs with carefully designed prompts and roles—within a collaborative workflow. By decomposing tasks and assigning subproblems to modules with dedicated tools and capabilities (e.g., planner, coder, critic), they can address complex problems such as web browsing, document processing, and multi-stage programming that exceed the scope of a single model. A central limitation, however, is that these systems are typically *training-free*: modules remain frozen pre-trained models orchestrated by handcrafted logic or prompting heuristics.

3 IN-THE-FLOW AGENTIC SYSTEM OPTIMIZATION

We aim to bridge the gap between trainable but monolithic reasoning models and flexible yet static agentic systems. We present AGENTFLOW, a flexible and trainable agentic system that integrates four specialized modules with an evolving memory (§3.1). Unlike prior agentic systems, AGENTFLOW directly optimizes the planner *within* the multi-turn loop of an agentic system (§3.2).

3.1 AGENTFLOW: AN IN-THE-FLOW AGENTIC SYSTEM

We propose AGENTFLOW, a general-purpose tool-integrated agentic framework for solving complex reasoning tasks through fine-grained planning and effective tool use within a multi-turn architecture. As shown in Figure 2, the framework comprises four specialized modules—**Action Planner** $\mathcal{P}$, **Tool Executor** $\mathcal{E}$, **Execution Verifier** $\mathcal{V}$, and **Solution Generator** $\mathcal{G}$ —coordinated by a shared evolving memory M and a toolset K . These modules interact sequentially and iteratively to perform *action planning*, *tool execution*, *context verification*, and *solution generation*, thereby enabling tool-integrated reasoning across multiple turns.

We formalize AGENTFLOW’s problem-solving process as a multi-turn Markov Decision Process (MDP). Given a query q and a toolset K , the system proceeds for a variable number of turns. Let M^t denote the memory state before turn t (with M^1 initialized from q). At turn t , the planner $\mathcal{P}$ (a trainable policy π_θ) formulates a sub-goal, selects an appropriate tool $k \in K$, and retrieves relevant context from memory, producing an action: $a^t \sim \pi_\theta(a^t \mid q, K, M^t)$.

The executor $\mathcal{E}$ invokes the chosen tool with context, yielding an execution observation $e^t \sim \mathcal{E}(e^t \mid a^t, K)$. The verifier $\mathcal{V}$ then evaluates whether e^t is valid and whether the accumulated memory is sufficient to solve the query, producing a binary verification signal $v^t \sim \mathcal{V}(v^t \mid q, e^t, M^t)$. If $v^t = 0$, the memory is updated deterministically to incorporate new evidence: $M^{t+1} = f_{\text{mem}}(M^t, a^t, e^t, v^t)$, where $f_{\text{mem}}(\cdot)$ denotes the memory-update function, which records agent-process information in a concise, structured form along with contextual details such as time, turn index, and error signals.

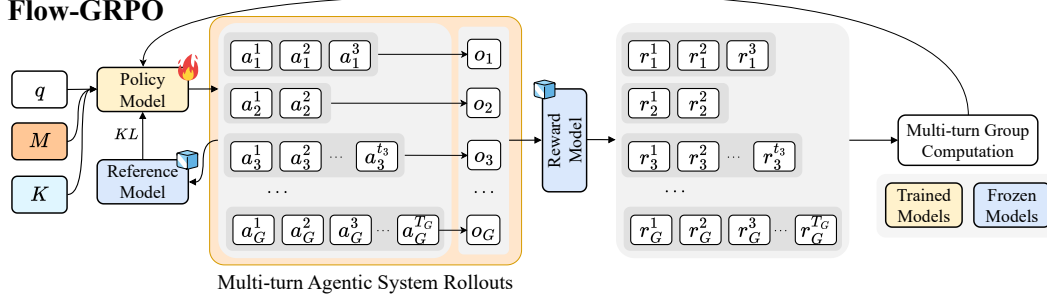


Figure 4: **Optimization for our proposed agentic system AGENTFLOW.** Given a query q , an evolving memory M , and a toolset K , the policy model generates actions that target sub-goals and select tools. It is trained via *Flow-based Group Refined Policy Optimization* (Flow-GRPO), which enables multi-turn reinforcement learning and stable optimization under collaborative dynamics.

The process repeats until $v^t = 1$ (termination) or a predefined maximum turn budget is reached. Upon termination at turn T , the solution generator $\mathcal{G}$ produces the final solution o , conditioned on the query and the accumulated memory: $o \sim \mathcal{G}(o \mid q, M^T)$.

This formulation decomposes multi-turn, tool-integrated reasoning into structured, observable transitions. After T turns, the trajectory $\tau = \{(a^t, e^t, v^t)\}_{t=1}^T$ records the history of planning, execution, and verification. The joint generative process can be written as

$$p_{\theta}(\{a^t, e^t, v^t\}_{t=1}^T, o \mid q) = \left[\prod_{t=1}^T \pi_{\theta}(a^t \mid q, K, M^t) \mathcal{E}(e^t \mid a^t, K) \mathcal{V}(v^t \mid q, e^t, M^t) \right] \mathcal{G}(o \mid q, M^T), \quad (2)$$

where $\{a^t, e^t, v^t\}_{t=1}^T$ are explicit realizations of the latent reasoning chain. Importantly, unlike latent thoughts behind trajectories, our memory M is an explicit and deterministic record of the reasoning process, ensuring transparency and controllability of multi-turn decisions.

3.2 IN-THE-FLOW REINFORCEMENT LEARNING OPTIMIZATION

We target tool-integrated *agentic systems* operating under *long-horizon* tasks with *sparse* rewards. In this setting, the **Action Planner** (the trainable policy of AGENTFLOW) selects a *sequence* of interdependent actions while the state (q, K, M^t) evolves with tool results and verifier feedback. Conventional *offline* training—e.g., supervised fine-tuning or preference fine-tuning on curated traces—optimizes the planner *outside* the active loop (Motwani et al., 2024; Park et al., 2025). This decoupling prevents real-time coordination with the executor, verifier, and solution generator, induces distribution shift between training and deployment, and provides limited guidance about *which* intermediate decisions truly matter. As a result, planners often adapt poorly to multi-turn dynamics; early errors cascade, and post-hoc fixes are brittle.

In-the-flow learning. To address these issues, we optimize the planner *in the flow* of execution. We roll out the full AGENTFLOW system under the current policy, collect the actual trajectory τ of states, actions, and tool events it induces, and update the policy within the agentic system using a verifiable final-outcome signal. This exposes the multi-turn credit-assignment problem directly and trains the planner on the exact states it will face at inference. Our objective, Flow-GRPO, is designed to stabilize learning under sparse, trajectory-level rewards over multiple turns.

As established in §3.1, rollouts in AGENTFLOW define a finite-horizon MDP with a variable horizon T . At turn t , the planner observes the state (q, K, M^t) , selects an action a^t , the executor and verifier return (e^t, v^t) , and the memory updates deterministically to M^{t+1} .

Policy optimization objective. The planner policy π_{θ} is trained to maximize the expected return over on-policy rollouts. Let $R(\tau)$ be the reward for a complete trajectory τ . The objective is:

$$\mathcal{J}(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}}[R(\tau)], \quad \theta^* = \arg \max_{\theta} \mathcal{J}(\theta), \quad (3)$$

where a rollout τ is the sequence of decisions $\{a^t\}_{t=1}^T$ generated on-policy by π_{θ} .

Final-outcome reward. Assigning credit to intermediate actions is challenging because each a^t influences the final solution only indirectly, and their value may only emerge after several turns (e.g.,

error or improvement accumulation). To avoid brittle local feedback, we adopt a *final-outcome-based reward*: every action within a rollout receives the same global reward signal, based on the correctness of the final solution o with respect to query q and ground truth y^* :

$$r = R(a^t) = \bar{R}(o, q, y^*), \quad \forall t = 1, \dots, T, \quad (4)$$

where $\bar{R}(o, q, y^*) \in \{0, 1\}$ is assigned by an LLM-as-judge rubric for semantic, numeric, and option-level equivalence (see §E.3). This propagates a trajectory-level success signal back through the reasoning chain, aligning every decision a^t with global correctness.

Objective function. We formalize **Flow-based Group Refined Policy Optimization** for the planner. The goal is to optimize the policy π_θ by maximizing the expected return over a group of parallel rollouts. For each query-label pair from training corpus $(q, y^*) \sim \mathcal{D}$, we sample a group of G on-policy trajectories $\{\tau_i\}_{i=1}^G$ by running the current behavior policy $\pi_{\theta_{\text{old}}}$ inside AGENTFLOW, where $\tau_i = \{a_i^1, \dots, a_i^{T_i}, o_i\}$. Let $s_i^t = (q, K, M_i^t)$ be the state at turn t of rollout i , a_i^t the planner’s action (a token sequence of length $|a_i^t|$), and o_i the final response. This structure is key to addressing the long-horizon credit assignment challenge: by broadcasting a single trajectory-level reward to all turns, we effectively decompose the *multi-turn RL* problem into a *set of independent, single-turn* policy updates; we provide a formal proof of this equivalence and analyze its convergence properties in §B. Each update for an action a_i^t is conditioned on the full historical context encapsulated in the state s_i^t and receives the same global success signal, simplifying optimization. The objective is

$$\mathcal{J}_{\text{Flow-GRPO}}(\theta) = \mathbb{E}_{(q, y^*) \sim \mathcal{D}, \{\tau_i\}_{i=1}^G \sim \pi_{\theta_{\text{old}}}} \left[\frac{1}{G} \sum_{i=1}^G \frac{1}{T_i} \sum_{t=1}^{T_i} \frac{1}{|a_i^t|} \sum_{j=1}^{|a_i^t|} \min \left\{ \rho_{i,j}^t A_i^t, \text{clip}(\rho_{i,j}^t, 1 - \epsilon, 1 + \epsilon) A_i^t \right\} - \beta \mathbb{D}_{\text{KL}}(\pi_\theta \parallel \pi_{\text{ref}}) \right], \quad (5)$$

where T_i is the (variable) number of turns in rollout i , and

$$\rho_{i,j}^t = \frac{\pi_\theta(a_{i,j}^t \mid s_i^t, a_{i,1:j-1}^t)}{\pi_{\theta_{\text{old}}}(a_{i,j}^t \mid s_i^t, a_{i,1:j-1}^t)} \quad (6)$$

is the token-level importance ratio for the j -th token of a_i^t , $\epsilon > 0$ is the PPO clipping parameter, and $\beta > 0$ controls the KL penalty to a fixed reference policy π_{ref} .

Group-normalized advantages. Because the reward in Eq. 4 is a single trajectory-level signal, the per-turn advantage A_i^t is constant over t within a rollout i . We reduce variance and sharpen credit assignment across the group by using a *group-normalized* advantage:

$$A_i^t = \frac{\bar{R}(o_i, q, y^*) - \text{mean}(\{\bar{R}(o_k, q, y^*)\}_{k=1}^G)}{\text{std}(\{\bar{R}(o_k, q, y^*)\}_{k=1}^G)}. \quad (7)$$

Technical contribution summary. To tackle *long-horizon, sparse-reward* training in multi-module agentic systems, we propose Flow-GRPO. This novel algorithm (i) formalizes the multi-turn RL problem in agentic systems into a series of *tractable, single-turn policy updates*, and (ii) *broadcasts* a single trajectory-level outcome to every turn to align local planner decisions with global success. Training uses an LLM-based rubric to assign verifiable final-outcome rewards, with group-normalized advantages, KL regularization, and clipping to stabilize learning.

4 EXPERIMENTS

4.1 EXPERIMENTAL SETUP

Implementation. In our main experiments, all modules—Action Planner, Tool Executor, Executive Verifier, and Solution Generator—are instantiated with the Qwen2.5-7B-Instruct model (Yang et al., 2024a). Among these, only the Action Planner is trainable. The system operates with five interactive tools: *Base Generator* is an instance of Qwen2.5-7B-Instruct that acts as the default reasoning engine if the planner decides not to use an external tool; *Python Coder* generates and executes Python code given a query and returns the execution result; *Google Search* searches the web

Model	Size	Search Intensive					Agentic		
		Bamboogle	2Wiki	HotpotQA	Musique	Avg.	Δ	GAIA	Δ
Qwen-2.5-7B-Instruct	7B-Inst	12.0	23.0	21.0	6.0	15.5	$\uparrow 41.8$	3.2	$\uparrow 29.9$
Qwen-2.5-14B-Instruct	14B-Inst	21.6	26.7	20.0	8.0	19.1	$\uparrow 38.2$	5.5	$\uparrow 27.6$
Qwen-2.5-32B-Instruct	32B-Inst	24.0	26.7	27.0	6.0	20.9	$\uparrow 36.4$	9.5	$\uparrow 23.6$
Llama-3.3-70B-Instruct	70B-Inst	18.4	22.7	52.0	16.0	27.3	$\uparrow 30.0$	3.2	$\uparrow 29.9$
GPT-4o-mini (Hurst et al., 2024)	~8B	40.8	35.6	41.0	15.0	33.1	$\uparrow 24.2$	7.1	$\uparrow 26.0$
GPT-4o (Hurst et al., 2024)	~200B	68.8	49.5	54.0	24.0	49.1	$\uparrow 8.2$	17.3	$\uparrow 15.8$
Supervised Fine-Tuning (SFT)	7B-Inst	12.0	25.9	22.0	6.6	16.6	$\uparrow 40.7$	3.2	$\uparrow 29.9$
Iter-RetGen (Shao et al., 2023)	7B-Inst	36.8	33.6	37.4	17.8	31.4	$\uparrow 25.9$	3.9	$\uparrow 29.2$
Search-R1 (Jin et al., 2025)	7B-Inst	43.2	38.2	37.0	14.6	33.3	$\uparrow 24.0$	19.1	$\uparrow 14.0$
ZeroSearch (Sun et al., 2025)	7B-Base	27.8	35.2	34.6	18.0	28.9	$\uparrow 28.4$	16.5	$\uparrow 16.6$
ReSearch (Chen et al., 2025)	7B-Base	42.4	47.6	43.5	22.3	39.0	$\uparrow 18.3$	17.3	$\uparrow 15.8$
StepSearch (Wang et al., 2025d)	7B-Base	40.0	36.6	38.6	22.6	34.5	$\uparrow 22.8$	—	—
VeriTool (Jiang et al., 2025)	7B-Base	46.4	45.3	44.8	19.3	39.0	$\uparrow 18.3$	11.2	$\uparrow 21.9$
AutoGen (Wu et al., 2024)	7B-Inst	59.6	44.0	50.0	15.9	42.4	$\uparrow 14.9$	6.3	$\uparrow 26.8$
AGENTFLOW	7B-Inst	58.4	60.0	51.3	19.2	47.2	$\uparrow 12.1$	17.2	$\uparrow 15.9$
AGENTFLOW (w/ Flow-GRPO)	7B-Inst	69.6	77.2	57.0	25.3	57.3	—	33.1	—

Table 1: **Accuracy comparison on search-intensive and agentic tasks.** 7B-Base refers to Qwen-2.5-7B-Base and 7B-Inst refers to Qwen-2.5-7B-Instruct. AutoGen and our AGENTFLOW method are agentic systems, which use Qwen-2.5-7B-Instruct for the LLM-powered agents and tools for fair comparison. We visualize the gains of AGENTFLOW to the each baseline in the Δ columns.

Model	Size	Math Reasoning					Scientific Reasoning			
		AIME24	AMC23	GameOf24	Avg.	Δ	GPQA	MedQA	Avg.	Δ
Qwen-2.5-7B-Instruct	7B-Inst	6.7	47.5	33.0	29.1	$\uparrow 22.5$	34.0	66.0	50.0	$\uparrow 13.5$
Qwen-2.5-14B-Instruct	14B-Inst	6.7	60.0	25.0	30.6	$\uparrow 21.0$	31.0	75.0	53.0	$\uparrow 10.5$
Llama-3.3-70B-Instruct	70B-Inst	6.7	47.5	31.0	28.4	$\uparrow 23.1$	35.0	67.0	51.0	$\uparrow 12.5$
Llama-3.1-405B-Instruct	405B-Inst	26.7	47.5	23.0	32.4	$\uparrow 19.1$	30.0	62.0	46.0	$\uparrow 17.5$
GPT-4o-mini (Hurst et al., 2024)	~8B	13.3	57.5	16.0	28.9	$\uparrow 22.6$	27.0	66.0	46.5	$\uparrow 17.0$
GPT-4o (Hurst et al., 2024)	~200B	13.3	60.0	32.0	35.1	$\uparrow 16.4$	31.0	60.0	45.5	$\uparrow 18.0$
Supervised Fine-Tuning (SFT)	7B-Inst	6.7	47.5	33.0	29.1	$\uparrow 22.5$	34.0	66.0	50.0	$\uparrow 13.5$
SimpleRL-reason (Zeng et al., 2025b)	7B-Base	16.7	60.0	33.0	36.6	$\uparrow 15.0$	45.0	65.0	50.0	$\uparrow 13.5$
Open-Reasoner-Zero (Hu et al., 2025a)	7B-Base	16.7	54.9	32.0	34.5	$\uparrow 17.0$	34.0	54.0	44.0	$\uparrow 19.5$
General-Reasoner (Ma et al., 2025)	7B-Base	13.3	55.0	33.0	33.8	$\uparrow 17.7$	35.5	61.0	48.3	$\uparrow 15.2$
Luffy (Yan et al., 2025)	7B-Inst	30.7	44.8	33.0	36.2	$\uparrow 15.3$	34.0	77.0	55.5	$\uparrow 8.0$
TIR (Yang et al., 2024b)	7B-Inst	10.0	50.0	33.0	31.0	$\uparrow 20.5$	42.0	76.8	59.4	$\uparrow 4.1$
ToRL (Li et al., 2025b)	7B-Inst	20.0	60.0	31.0	37.0	$\uparrow 14.5$	35.0	76.5	55.8	$\uparrow 7.7$
AutoGen (Wu et al., 2024)	7B-Inst	13.3	57.5	24.0	31.6	$\uparrow 19.9$	42.0	72.0	57.0	$\uparrow 6.5$
AGENTFLOW	7B-Inst	16.7	47.4	31.0	31.7	$\uparrow 19.8$	37.0	76.0	56.5	$\uparrow 7.0$
AGENTFLOW (w/ Flow-GRPO)	7B-Inst	40.0	61.5	53.0	51.5	—	47.0	80.0	63.5	—

Table 2: **Accuracy comparison of mathematical and scientific reasoning tasks.** As the same in Table 1, AutoGen and AGENTFLOW use Qwen-2.5-7B-Instruct for the LLM-powered tools.

and returns a summarization of Top-K search results; *Wikipedia Search* searches articles matching a given query and returns a summarization; and *Web Search* returns summarized information from a given web page. During the RL fine-tuning phase, we mix data from Search-R1 (Jin et al., 2025) and DeepMath (He et al., 2025) as training data, which provides paired question-answer examples across search and mathematical domains.

Training. We provide further details on the training setup for AGENTFLOW. Our Flow-GRPO implementation uses a learning rate of 1×10^{-6} . The Action Planner generates actions with a sampling temperature of 0.5 to balance exploration and exploitation. To prevent policy collapse and stabilize training, we incorporate a KL-divergence penalty against a reference policy with a coefficient $\beta = 0.001$. The maximum output length for the planner is set to 2048 tokens to ensure complete exploration during rollouts. We use a batch size of 32 with 8 rollouts per sample.

To accelerate the training speed, we limit the maximum number of turns per rollout to 3. The final-outcome reward signal (Eq. 4) is provided by an LLM-as-judge, for which we use GPT-4o. All tool calls are executed synchronously with a 500-second timeout to handle external service latency robustly. The LLM engines within the tools are set to a temperature of 0.0 to ensure deterministic and stable outputs. The full training process was conducted on 8 NVIDIA A100 GPUs. Further details on agent prompts and the memory update mechanism are provided in §E.1.

Evaluation. To comprehensively evaluate tool-use capabilities of AGENTFLOW, we conduct experiments on four types of reasoning tasks: (1) *Knowledge-intensive search* including Bam-

boogle (Press et al., 2023), 2Wiki (Ho et al., 2020), HotpotQA (Yang et al., 2018), and Musique (Trivedi et al., 2022); (2) *Agentic reasoning* such as GAIA (Mialon et al., 2023) (where we adopt the textual split); (3) *Logic-dense mathematical reasoning* including AIME2024 (Art of Problem Solving, 2025), AMC23 (MAA, 2023), and GameOf24 (Lightman et al., 2023); and (4) *Scientific reasoning* including GPQA (Rein et al., 2024) and MedQA (Yang et al., 2024c). To mitigate randomness, we report the average accuracy across three trials for all experiments. More evaluation details are provided in §C.

4.2 MAIN RESULTS

Baselines. As presented in Tables 1 and 2, we include five categories of baselines: (1) *Open-source LLMs*: Qwen2.5 (Yang et al., 2024a), Llama-3.1, and Llama-3.3 (Dubey et al., 2024); (2) *Proprietary LLMs*: GPT-4o-mini and GPT-4o; (3) *Reasoning LLMs*: supervised fine-tuning (Yang et al., 2024b), SimpleRL-reason, Open-Reasoner-Zero, General-Reasoner, and LUFFY; (4) *Tool-integrated reasoning LLMs*: both search-enhanced, including Iter-RetGen, Search-R1, ZeroSearch, ReSearch, StepSearch, and VeriTool, and code-enhanced, including TIR and ToRL; (5) *Training-free agentic system*: AutoGen. More details on baseline implementations are in §C.2.

Key insights. AGENTFLOW consistently outperforms all baseline models by large margins. Compared to the best-performing 7B models without tool integration, AGENTFLOW achieves absolute gains of 40.7% on search (SFT), 29.9% on agentic reasoning (SFT), 15.0% on math (SimpleRL-reason), and 8.0% on scientific tasks (Luffy). Against specialized tool-integrated systems, AGENTFLOW surpasses the top models by 14.9% in search (AutoGen), 14.0% in agentic reasoning (Search-R1), 14.5% in math (ToRL), and 4.1% in science (TIR). Notably, our 7B-backbone AGENTFLOW even outperforms the ~200B-parameter GPT-4o across all domains, with gains ranging from 8.2% to 18.0%. A detailed analysis is provided in §D.1.

4.3 IN-DEPTH ANALYSIS OF OPTIMIZED PLANNING

Flow-GRPO optimizes tool usage. We compare tool usage distributions before and after in-the-flow RL training. Figure 5 shows results on two knowledge-intensive tasks, 2Wiki and MedQA, which exhibit distinct optimization patterns alongside improved task accuracy. For 2Wiki, which requires broad factual knowledge, Flow-GRPO optimizes the planner to increase Google Search usage by 42.0%. In contrast, for the specialized MedQA benchmark, which requires deep, domain-specific information retrieval, fine-tuning shifts the planner away from general tools, reducing Google Search calls (66.2→10.9%) in favor of in-document Web Search (0→19.5%) and specialized Wikipedia Search (0→59.8%). This demonstrates that the planner learns to select task-appropriate tools.

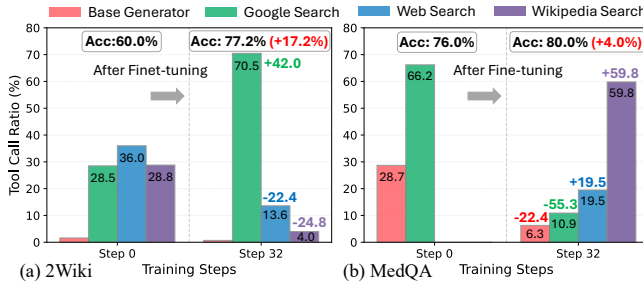


Figure 5: Tool call ratio change by Flow-GRPO fine-tuning.

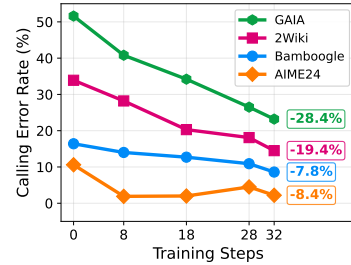


Figure 6: Calling error rate.

Flow-GRPO enhances tool-calling efficacy. A key aspect of the model’s improvement is its increased reliability in tool usage. As shown in Figure 6, the tool-calling error rate consistently decreases across tasks during training, with a reduction of up to 28.4% on GAIA. This trend indicates that the training process not only teaches the model *which* tool to use but also *how* to invoke it correctly with proper arguments and format, leading to more robust and effective tool integration.

Flow-GRPO incentivizes autonomous discovery of new solutions. We further examine qualitative examples in Figure 7 and additional cases in §F. These cases show that AGENTFLOW, trained with Flow-GRPO, develops enhanced capabilities for task planning and tool use. The planner exhibits adaptive efficiency, stronger self-correction, and spontaneous new integration of tools throughout step-by-step problem-solving, autonomously discovering effective solution pathways.

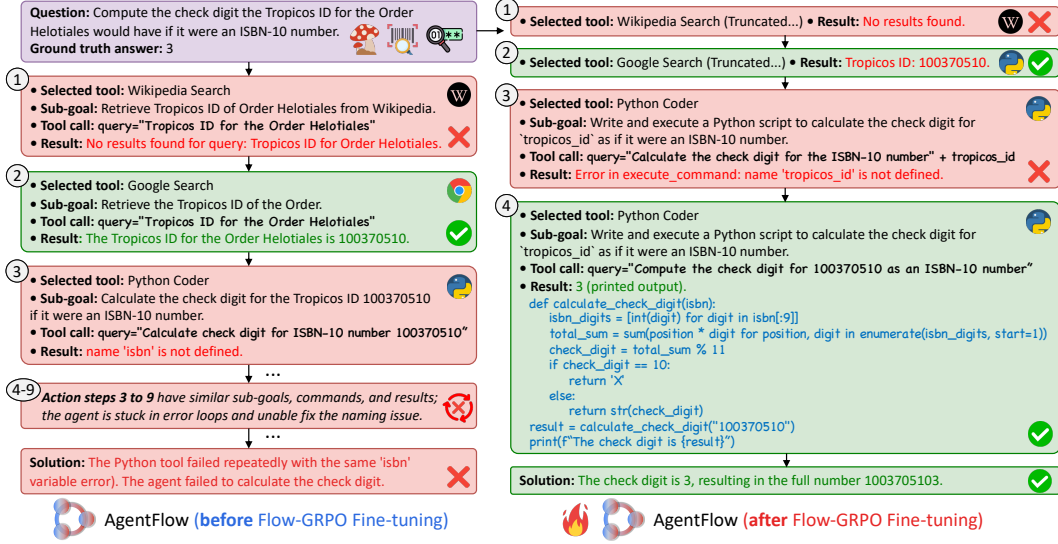


Figure 7: **One case study example.** Initially failed with repetitive errors (left), AGENTFLOW, trained with Flow-GRPO, explores a new solution pathway at turn 4 after two failed attempts (right).

4.4 TRAINING STRATEGIES ON THE PLANNER

We conduct an ablation study to analyze the impact of different training strategies for the Action Planner module in AGENTFLOW, with results reported in Table 3. The executor, verifier, and generator modules remain fixed as Qwen2.5-7B-Instruct, consistent with our main setup (§4.1).

Planner Model	Training	Bamboogle	2Wiki	GAIA	AIME24	AMC23	GameOf24	Avg.
Qwen-2.5-7B	Frozen	58.4	60.0	17.2	16.7	47.4	31.0	38.5
GPT-4o	Frozen	65.0 $\uparrow$ 6.6	70.0 $\uparrow$ 10.0	23.6 $\uparrow$ 6.4	16.7 $\uparrow$ 0.0	48.7 $\uparrow$ 1.3	42.0 $\uparrow$ 11.0	44.3 $\uparrow$ 5.8
Qwen-2.5-7B	SFT	30.4 $\downarrow$ 28.0	32.7 $\downarrow$ 27.3	6.3 $\downarrow$ 10.9	3.3 $\downarrow$ 13.4	37.5 $\downarrow$ 9.9	7.0 $\downarrow$ 24.0	19.5 $\downarrow$ 19.0
Qwen-2.5-7B	Flow-GRPO	69.6 $\uparrow$ 11.2	77.2 $\uparrow$ 17.2	33.1 $\uparrow$ 15.9	40.0 $\uparrow$ 23.3	61.5 $\uparrow$ 14.1	53.0 $\uparrow$ 22.0	55.7 $\uparrow$ 17.2

Table 3: Performance comparison of AGENTFLOW across different training methods.

A more capable planner is beneficial, but has limits. Replacing the frozen Qwen2.5-7B-Instruct baseline with a stronger proprietary model, GPT-4o, yields only a modest 5.8% average gain. This indicates a key bottleneck that, while a more powerful model improves planning, its static nature prevents co-adaptation with the live dynamics of AGENTFLOW.

Offline SFT leads to performance collapse, while in-the-flow RL is crucial. The limitations of a static planner are further exposed when distilling GPT-4o’s behavior via offline supervised fine-tuning (SFT) on its trajectories as Action Planner in AGENTFLOW. This results in a catastrophic performance collapse, with an average accuracy drop of 19.0% compared to the frozen baseline. This failure arises from the token-level imitation objective of SFT, which misaligns with trajectory-level task success and prevents the planner from adapting to dynamic tool feedback or recovering from compounding errors. In contrast, training the planner with our on-policy Flow-GRPO method proves highly effective: by optimizing for the final outcome, the planner learns to handle long-horizon workflows, achieving a 17.2% average gain over the frozen baseline.

4.5 TRAINING EFFICIENCY ANALYSIS

Optimized planning with increased rewards and condensed responses.

We analyze the training dynamics of the AGENTFLOW planner by tracking its average reward and response length on the train set (Figure 8a). Training rewards steadily increase, indicating effective policy improvement via Flow-GRPO. Meanwhile, response length, after an initial ex-

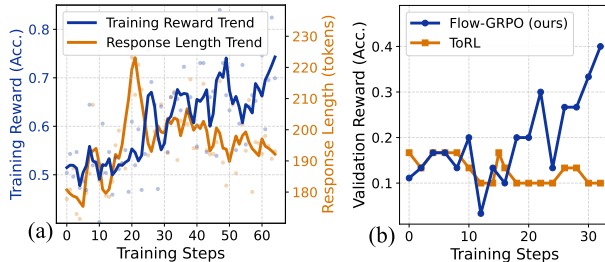


Figure 8: Training dynamics and efficiency of Flow-GRPO.

ploratory rise, progressively shortens and stabilizes. This shows the planner learns to balance conciseness and informativeness, avoiding unnecessarily long outputs.

Flow-GRPO efficiency over tool-integrated reasoning RL. We compare AGENTFLOW (trained with Flow-GRPO) against a monolithic tool-integrated reasoning baseline (ToRL) on AIME24. As shown in Figure 8b, AGENTFLOW achieves sustained performance gains, with validation accuracy growing steadily. In contrast, ToRL’s performance quickly stagnates and trends downwards, highlighting the superior efficiency of our agentic training approach, which uses decomposition and stable credit assignment to avoid the instability.

4.6 SCALING TRENDS IN AGENTFLOW

Training scaling in backbone size.

We study how backbone LLM scale affects AGENTFLOW’s performance and the efficacy of Flow-GRPO. We build two versions of the system: one using Qwen2.5-3B-Instruct and another using Qwen2.5-7B-Instruct for all four modules (planner, executor, verifier, and generator) and tools. In both, only the planner is fine-tuned with Flow-GRPO. As shown in Figure 9, Flow-GRPO fine-tuning consistently improves performance across tasks for both backbones. This demonstrates that our in-the-flow optimization is effective across model capacities, enhancing AGENTFLOW regardless of LLM size.

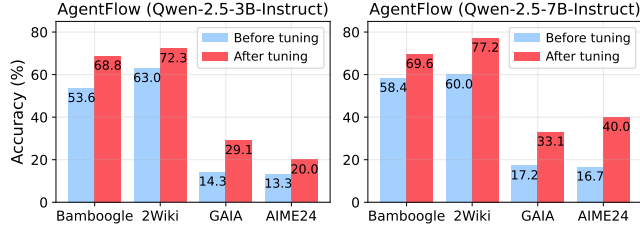


Figure 9: Flow-GRPO fine-tuning offers consistent gains on AGENTFLOW as the backbone model size scales from 3B to 7B.

Inference scaling in turn budgets. We investigate how the maximum allowed turns ($T_{\max}$) affect reasoning depth and final performance of AGENTFLOW during test-time inference with the Qwen2.5-7B-Instruct backbone. As shown in Figure 10, increasing $T_{\max}$ from 3 to 10 consistently improves outcomes across all tasks, accompanied by a rise in average turns consumed. On knowledge-intensive benchmarks such as 2Wiki and GAIA, a larger turn budget enables AGENTFLOW for deeper information retrieval. On mathematical benchmarks like GameOf24 and AIME24, it supports decomposed sub-goals, alternative strategies, and refinement of errors. Final performance peaks at $T_{\max} = 10$ for all tasks, confirming that a longer reasoning horizon benefits the system without causing degenerate loops. This validates that AGENTFLOW adapts its turn allocation to problem complexity to achieve better solutions through iterative refinement.

Turns ($T_{\max}$)	3	5	7	10
2Wiki	2.22	3.18	3.81	4.44
GameOf24	1.63	2.12	2.36	2.67
AIME24	1.63	1.63	1.86	1.90
GAIA	2.43	3.46	4.28	5.42

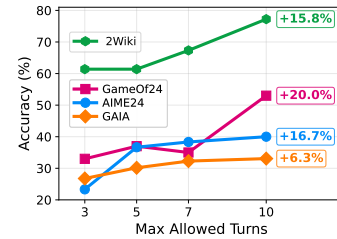


Figure 10: Average turns and accuracy with increased $T_{\max}$.

5 RELATED WORK

Reinforcement learning (RL) from outcome-based rewards has become a dominant paradigm for training LLMs to use external tools. Much of this work trains a single, monolithic policy to interleave reasoning with tool calls. This strategy has proven effective in specialized, single-tool settings, such as code execution for mathematical problems (Mai et al., 2025; Xue et al., 2025; Feng et al., 2025; Li et al., 2025b) and web search for knowledge-intensive questions (Chen et al., 2025; Jin et al., 2025; Song et al., 2025; Li et al., 2025a; Sun et al., 2025). Recent efforts have extended this monolithic framework to multi-tool environments by focusing on data synthesis (Dong et al., 2025), unified training infrastructure (Jiang et al., 2025), and principled reward design (Qian et al., 2025a; Zhang et al., 2025). However, this monolithic approach scales poorly as task complexity and planning horizons grow. The central challenge is long-horizon credit assignment; attributing a final outcome to specific intermediate tool calls remains difficult, even with fine-grained, turn-level rewards (Zeng et al., 2025a; Wang et al., 2025d). This difficulty leads to training instability and brittle inference-time generalization, manifesting as strategic deficiencies like tool overuse or “cognitive offloading” (Wang et al., 2025b; Qian et al., 2025b), suboptimal personalization (Cheng et al., 2025), and poor alignment with user preferences for tool invocation (Huang et al., 2025).

Agentic systems with tool use. Agentic systems offer an alternative to monolithic models by decomposing tasks across specialized modules. Many such systems are training-free, orchestrating pre-trained LLMs with handcrafted logic and prompting, as seen in frameworks like AutoGen (Wu et al., 2024), MetaGPT (Hong et al., 2024), and OctoTools (Lu et al., 2025). This static approach, however, limits their ability to learn and adapt collaborative strategies from experience. Recognizing this, recent work explores training these systems to improve coordination (Deng et al., 2025; Liao et al., 2025). However, most training paradigms are *offline*, relying on supervised fine-tuning or preference optimization on static datasets (Motwani et al., 2024; Park et al., 2025). These methods are decoupled from the live, multi-turn dynamics of the system, preventing modules from learning to adapt to evolving tool outputs or recover from early mistakes. Training directly *in the flow* with on-policy RL is difficult due to sparse rewards and long-horizon credit assignment, where feedback is delayed across long reasoning chains and shifting state distributions (Wang et al., 2025c). Consequently, these systems often suffer from brittle adaptation and require complex reward shaping to learn effectively (Wang et al., 2025a).

6 CONCLUSION

We presented AGENTFLOW, a trainable, *in-the-flow* agentic system that coordinates four specialized modules via an evolving memory and optimizes its planner directly *inside* the multi-turn loop. To enable stable on-policy learning under long-horizon, sparse-reward settings, we introduced Flow-GRPO, which *converts* multi-turn RL into a sequence of tractable *single-turn* policy updates by *broadcasting* a single, verifiable trajectory-level outcome to every turn and stabilizing credit assignment with group-normalized advantages. Comprehensive experiments show that AGENTFLOW achieves strong cross-domain performance, surpassing specialized baselines and even larger proprietary models. In-depth analyses confirm improved planning and tool-calling reliability, along with positive scaling trends in model size and allowed turn budgets. Future research will focus on extending in-the-flow optimization to other modules, incorporating more fine-grained reward signals, and scaling the framework to tackle more complex, open-ended tasks.

ACKNOWLEDGMENT

We would like to thank Yihe Deng, Xuehang Guo, and Kunlun Zhu for their valuable input during the early stages of this work. We are grateful to Lambda for providing GPU resources. This work was partially supported by the Hoffman-Yee Research Grants program at Stanford HAI, the AI for Math Fund by Renaissance Philanthropy, ONR MURI N00014-24-1-2748, and the AI Research Hub Project through KAIST.

REFERENCES

- Art of Problem Solving. Aime problems and solutions, 2025. URL https://artofproblemsolving.com/wiki/index.php/AIME_Problems_and_Solutions. 8, 22
- Mingyang Chen, Tianpeng Li, Haoze Sun, Yijie Zhou, Chenzheng Zhu, Haofen Wang, Jeff Z Pan, Wen Zhang, Huajun Chen, Fan Yang, et al. ReSearch: Learning to reason with search for llms via reinforcement learning. *arXiv preprint arXiv:2503.19470*, 2025. 2, 4, 7, 10, 21
- Zihao Cheng, Hongru Wang, Zeming Liu, Yuhang Guo, Yuanfang Guo, Yunhong Wang, and Haifeng Wang. ToolSpectrum: Towards personalized tool utilization for large language models. In *Findings of the Association for Computational Linguistics: ACL 2025*, pp. 20679–20699, 2025. 10
- Yingfan Deng, Anhao Zhou, Yuan Yuan, Xian Zhang, Yifei Zou, and Dongxiao Yu. Pe-ma: Parameter-efficient co-evolution of multi-agent systems. *arXiv preprint arXiv:2506.11803*, 2025. 11
- Guanting Dong, Yifei Chen, Xiaoxi Li, Jiajie Jin, Hongjin Qian, Yutao Zhu, Hangyu Mao, Guorui Zhou, Zhicheng Dou, and Ji-Rong Wen. Tool-star: Empowering llm-brained multi-tool reasoner via reinforcement learning. *arXiv preprint arXiv:2505.16410*, 2025. 2, 10

-
- Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024. 8, 20
- Jiazhan Feng, Shijue Huang, Xingwei Qu, Ge Zhang, Yujia Qin, Baoquan Zhong, Chengquan Jiang, Jinxin Chi, and Wanjun Zhong. Retool: Reinforcement learning for strategic tool use in llms. *arXiv preprint arXiv:2504.11536*, 2025. 2, 4, 10
- Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025. 2
- Zhiwei He, Tian Liang, Jiahao Xu, Qiuzhi Liu, Xingyu Chen, Yue Wang, Linfeng Song, Dian Yu, Zhenwen Liang, Wenxuan Wang, et al. Deepmath-103k: A large-scale, challenging, decontaminated, and verifiable mathematical dataset for advancing reasoning. *arXiv preprint arXiv:2504.11456*, 2025. 7
- Xanh Ho, Anh-Khoa Duong Nguyen, Saku Sugawara, and Akiko Aizawa. Constructing a multi-hop qa dataset for comprehensive evaluation of reasoning steps. In *Proceedings of the 28th International Conference on Computational Linguistics (COLING)*, pp. 6609–6625, 2020. 8, 21
- Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, et al. MetaGPT: Meta programming for a multi-agent collaborative framework. In *International Conference on Learning Representations (ICLR)*, 2024. 2, 4, 11
- Jingcheng Hu, Yinmin Zhang, Qi Han, Daxin Jiang, Xiangyu Zhang, and Heung-Yeung Shum. Open-reasoner-zero: An open source approach to scaling up reinforcement learning on the base model. *arXiv preprint arXiv:2503.24290*, 2025a. 7, 20
- Mengkang Hu, Yuhang Zhou, Wendong Fan, Yuzhou Nie, Bowei Xia, Tao Sun, Ziyu Ye, Zhaoxuan Jin, Yingru Li, Qiguang Chen, et al. Owl: Optimized workforce learning for general multi-agent assistance in real-world task automation. *arXiv preprint arXiv:2505.23885*, 2025b. 2
- Chengrui Huang, Shen Gao, Zhengliang Shi, Dongsheng Wang, and Shuo Shang. TTPA: Token-level tool-use preference alignment training framework with fine-grained evaluation. *arXiv preprint arXiv:2505.20016*, 2025. 10
- Aaron Hurst, Adam Lerer, Adam P Goucher, Adam Perelman, Aditya Ramesh, Aidan Clark, AJ Ostrow, Akila Welihinda, Alan Hayes, Alec Radford, et al. GPT-4o system card. *arXiv preprint arXiv:2410.21276*, 2024. 3, 7, 20
- Dongfu Jiang, Yi Lu, Zhuofeng Li, Zhiheng Lyu, Ping Nie, Haozhe Wang, Alex Su, Hui Chen, Kai Zou, Chao Du, et al. VerlTool: Towards holistic agentic reinforcement learning with tool use. *arXiv preprint arXiv:2509.01055*, 2025. 7, 10, 21
- Bowen Jin, Hansi Zeng, Zhenrui Yue, Jinsung Yoon, Sercan Arik, Dong Wang, Hamed Zamani, and Jiawei Han. Search-R1: Training llms to reason and leverage search engines with reinforcement learning. *arXiv preprint arXiv:2503.09516*, 2025. 2, 4, 7, 10, 20, 21
- Di Jin, Eileen Pan, Nassim Oufattole, Wei-Hung Weng, Hanyi Fang, and Peter Szolovits. What disease does this patient have? a large-scale open domain question answering dataset from medical exams. *Applied Sciences*, 11(14):6421, 2021. 22
- Xiaoxi Li, Guanting Dong, Jiajie Jin, Yuyao Zhang, Yujia Zhou, Yutao Zhu, Peitian Zhang, and Zhicheng Dou. Search-o1: Agentic search-enhanced large reasoning models. *arXiv preprint arXiv:2501.05366*, 2025a. 10
- Xuefeng Li, Haoyang Zou, and Pengfei Liu. ToRL: Scaling tool-integrated rl. *arXiv preprint arXiv:2503.23383*, 2025b. 7, 10, 21
- Junwei Liao, Muning Wen, Jun Wang, and Weinan Zhang. Marft: Multi-agent reinforcement fine-tuning. *arXiv preprint arXiv:2504.16129*, 2025. 11

-
- Hunter Lightman, Vineet Kosaraju, Yuri Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let’s verify step by step. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2023. 8
- Nathan Lile. Math twenty four (24s game) dataset. <https://huggingface.co/datasets/nlile/24-game>, 2024. 22
- Pan Lu, Bowen Chen, Sheng Liu, Rahul Thapa, Joseph Boen, and James Zou. OctoTools: An agentic framework with extensible tools for complex reasoning. *arXiv preprint arXiv:2502.11271*, 2025. 4, 11, 20
- Xueguang Ma, Qian Liu, Dongfu Jiang, Ge Zhang, Zejun Ma, and Wenhua Chen. General-reasoner: Advancing llm reasoning across all domains. *arXiv preprint arXiv:2505.14652*, 2025. 7, 20
- MAA. American mathematics competitions. In *American Mathematics Competitions*, 2023. 8, 22
- Xinji Mai, Haotian Xu, Xing W, Weinong Wang, Yingying Zhang, and Wenqiang Zhang. Agent RL Scaling Law: Agent RL with Spontaneous Code Execution for Mathematical Problem Solving. *arXiv preprint arXiv:2505.07773*, 2025. 2, 10
- Grégoire Mialon, Clémentine Fourrier, Thomas Wolf, Yann LeCun, and Thomas Scialom. Gaia: a benchmark for general ai assistants. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2023. 8, 22
- Moonshot AI. Kimi-Researcher: End-to-End RL Training for Emerging Agentic Capabilities. <https://moonshotai.github.io/Kimi-Researcher/>, June 2025. 2
- Sumeet Ramesh Motwani, Chandler Smith, Rocktim Jyoti Das, Rafael Rafailov, Ivan Laptev, Philip HS Torr, Fabio Pizzati, Ronald Clark, and Christian Schroeder de Witt. Malt: Improving reasoning with multi-agent llm training. *arXiv preprint arXiv:2412.01928*, 2024. 2, 5, 11
- Chanwoo Park, Seungju Han, Xingzhi Guo, A. Ozdaglar, Kaiqing Zhang, and Joo-Kyung Kim. MA-PoRL: Multi-agent post-co-training for collaborative large language models with reinforcement learning. In *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2025. URL <https://api.semanticscholar.org/CorpusId:276580906>. 2, 5, 11
- Ofir Press, Muru Zhang, Sewon Min, Ludwig Schmidt, Noah A Smith, and Mike Lewis. Measuring and narrowing the compositionality gap in language models. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pp. 5687–5711, 2023. 8, 21
- Cheng Qian, Emre Can Acikgoz, Qi He, Hongru Wang, Xiusi Chen, Dilek Hakkani-Tür, Gokhan Tur, and Heng Ji. ToolRL: Reward is all tool learning needs. *arXiv preprint arXiv:2504.13958*, 2025a. 2, 4, 10
- Cheng Qian, Emre Can Acikgoz, Hongru Wang, Xiusi Chen, Avirup Sil, Dilek Hakkani-Tür, Gokhan Tur, and Heng Ji. SMART: Self-aware agent for tool overuse mitigation. In *Findings of the Association for Computational Linguistics: ACL 2025*, pp. 4604–4621, 2025b. 10
- David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R Bowman. Gpqa: A graduate-level google-proof q&a benchmark. In *First Conference on Language Modeling*, 2024. 8, 22
- John Schulman, Sergey Levine, Pieter Abbeel, Michael Jordan, and Philipp Moritz. Trust region policy optimization. In *International Conference on Machine Learning (ICML)*, pp. 1889–1897. PMLR, 2015. 19
- Zhihong Shao, Yeyun Gong, Yelong Shen, Minlie Huang, Nan Duan, and Weizhu Chen. Enhancing retrieval-augmented large language models with iterative retrieval-generation synergy. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pp. 9248–9274, 2023. 7, 21
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Yang Wu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024. 3

-
- Huatong Song, Jinhao Jiang, Yingqian Min, Jie Chen, Zhipeng Chen, Wayne Xin Zhao, Lei Fang, and Ji-Rong Wen. R1-searcher: Incentivizing the search capability in llms via reinforcement learning. *arXiv preprint arXiv:2503.05592*, 2025. 2, 10
- Hao Sun, Zile Qiao, Jiayan Guo, Xuanbo Fan, Yingyan Hou, Yong Jiang, Pengjun Xie, Yan Zhang, Fei Huang, and Jingren Zhou. Zerossearch: Incentivize the search capability of llms without searching. *arXiv preprint arXiv:2505.04588*, 2025. 7, 10, 21
- Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. Musique: Multihop questions via single-hop question composition. *Transactions of the Association for Computational Linguistics (TACL)*, 10:539–554, 2022. 8, 21
- Hanlin Wang, Chak Tou Leong, Jiashuo Wang, Jian Wang, and Wenjie Li. SPA-RL: Reinforcing llm agents via stepwise progress attribution. *arXiv preprint arXiv:2505.20732*, 2025a. 11
- Hongru Wang, Cheng Qian, Wanjun Zhong, Xiusi Chen, Jiahao Qiu, Shijue Huang, Bowen Jin, Mengdi Wang, Kam-Fai Wong, and Heng Ji. Acting less is reasoning more! teaching model to act efficiently. *arXiv preprint arXiv:2504.14870*, 2025b. URL <https://arxiv.org/pdf/2504.14870>. 10
- Zihan Wang, Kangrui Wang, Qineng Wang, Pingyue Zhang, Linjie Li, Zhengyuan Yang, Xing Jin, Kefan Yu, Minh Nhat Nguyen, Licheng Liu, et al. RAGEN: Understanding self-evolution in llm agents via multi-turn reinforcement learning. *arXiv preprint arXiv:2504.20073*, 2025c. 2, 11
- Ziliang Wang, Xuhui Zheng, Kang An, Cijun Ouyang, Jialu Cai, Yuhang Wang, and Yichao Wu. Stepsearch: Igniting llms search ability via step-wise proximal policy optimization. *arXiv preprint arXiv:2505.15107*, 2025d. 7, 10, 21
- Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, et al. Autogen: Enabling next-gen llm applications via multi-agent conversations. In *First Conference on Language Modeling (COLM)*, 2024. 2, 4, 7, 11, 21
- Zhenghai Xue, Longtao Zheng, Qian Liu, Yingru Li, Xiaosen Zheng, Zejun Ma, and Bo An. Simpletir: End-to-end reinforcement learning for multi-turn tool-integrated reasoning. *arXiv preprint arXiv:2509.02479*, 2025. 2, 10
- Jianhao Yan, Yafu Li, Zican Hu, Zhi Wang, Ganqu Cui, Xiaoye Qu, Yu Cheng, and Yue Zhang. Learning to reason under off-policy guidance. *arXiv preprint arXiv:2504.14945*, 2025. 7, 20
- An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiayi Yang, Jingren Zhou, Junyang Lin, Kai Dang, Keming Lu, Keqin Bao, Kexin Yang, Le Yu, Mei Li, Mingfeng Xue, Pei Zhang, Qin Zhu, Rui Men, Runji Lin, Tianhao Li, Tianyi Tang, Tingyu Xia, Xingzhang Ren, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yu Wan, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, and Zihan Qiu. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024a. 2, 6, 8, 20
- An Yang, Beichen Zhang, Binyuan Hui, Bofei Gao, Bowen Yu, Chengpeng Li, Dayiheng Liu, Jianhong Tu, Jingren Zhou, Junyang Lin, et al. Qwen2.5-math technical report: Toward mathematical expert model via self-improvement. *arXiv preprint arXiv:2409.12122*, 2024b. 7, 8, 21
- Hang Yang, Hao Chen, Hui Guo, Yineng Chen, Ching-Sheng Lin, Shu Hu, Jinrong Hu, Xi Wu, and Xin Wang. Llm-medqa: Enhancing medical question answering through case studies in large language models. *arXiv preprint arXiv:2501.05464*, 2024c. 8
- Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William Cohen, Ruslan Salakhutdinov, and Christopher D Manning. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 2369–2380, 2018. 8, 21
- Qiyang Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Weinan Dai, Tiantian Fan, Gaohong Liu, Lingjun Liu, et al. Dapo: An open-source llm reinforcement learning system at scale. *arXiv preprint arXiv:2503.14476*, 2025. 3

-
- Siliang Zeng, Quan Wei, William Brown, Oana Frunza, Yuriy Nevmyvaka, and Mingyi Hong. Reinforcing multi-turn reasoning in llm agents via turn-level credit assignment. *arXiv preprint arXiv:2505.11821*, 2025a. [10](#)
- Weihao Zeng, Yuzhen Huang, Qian Liu, Wei Liu, Keqing He, Zejun Ma, and Junxian He. Simplerl-zoo: Investigating and taming zero reinforcement learning for open base models in the wild. *arXiv preprint arXiv:2503.18892*, 2025b. [2](#), [7](#), [20](#)
- Shaokun Zhang, Yi Dong, Jieyu Zhang, Jan Kautz, Bryan Catanzaro, Andrew Tao, Qingyun Wu, Zhiding Yu, and Guilin Liu. Nemotron-research-tool-n1: Tool-using language models with reinforced reasoning. *arXiv preprint arXiv:2505.00024*, 2025. [2](#), [10](#)

TABLE OF CONTENTS

A	Training Algorithm of AGENTFLOW	17
B	Theoretical Analysis of Flow-GRPO	18
B.1	Preliminaries and Notation	18
B.2	Equivalence Proof for Optimization Objectives	18
B.3	Convergence Analysis	19
C	Experimental Details	20
C.1	Evaluation Details	20
C.2	Compared Baselines	20
C.3	Evaluation Datasets	21
D	More Discussion about Experiment Results	22
D.1	Main Result Analysis	22
D.2	In-depth Analysis of Optimized Planning	23
E	Instruction Templates in AGENTFLOW	24
E.1	Modules and Memory	24
E.2	Toolset Metadata	27
E.3	LLM-based Judging	32
F	Case Studies	33
F.1	Example 1: Efficient Search for Simple Tasks	33
F.2	Example 2: Spontaneous Brute-force	34
F.3	Example 3: A Good Initial Plan is Essential	36
F.4	Example 4: Robust Self-Correction and Adaptation	38
F.5	Example 5: New Combo: Retrieve with Specific URL	40
F.6	Example 6: Rapid and Correct Physics Calculation	42
F.7	Example 7: Multi-Source Cross-Verification	44

A TRAINING ALGORITHM OF AGENTFLOW

We provide a flowchart of the overall training algorithm of AGENTFLOW (§3) in Algorithm 1.

Algorithm 1 In-the-Flow Optimization for AGENTFLOW

Require: Dataset $\mathcal{D}$, Action Planner policy π_θ , Tool Executor $\mathcal{E}$, Executive Verifier $\mathcal{V}$, Solution Generator $\mathcal{G}$, Toolset K , and Shared Evolving Memory M

Ensure: Optimized Action Planner parameters θ^*

```
1: for each training iteration do
2:   for each query–label pair  $(q, y^*) \sim \mathcal{D}$  do
3:     1. IN-THE-FLOW ROLLOUT GENERATION
4:     Initialize:  $t \leftarrow 1, M^t \leftarrow q$ 
5:     repeat
6:        $a^t \sim \pi_\theta(a^t \mid q, K, M^t)$  {Plan Action}
7:        $e^t \sim \mathcal{E}(e^t \mid a^t, K)$  {Execute Action}
8:        $v^t \sim \mathcal{V}(v^t \mid q, e^t, M^t)$  {Verify Result}
9:        $M^{t+1} = f_{\text{mem}}(M^t, a^t, e^t, v^t)$  {Update Memory}
10:       $t \leftarrow t + 1$ 
11:    until termination condition met
12:     $o \sim \mathcal{G}(o \mid q, M^T)$  {Generate Final Solution}
13:    2. REWARD COMPUTATION
14:     $R(a^t) = \bar{R}(o, q, y^*), \quad \forall t = 1, \dots, T$ 
15:    3. POLICY UPDATE
16:    Update the Action Planner policy  $\pi_\theta$  by maximizing the Flow-GRPO objective (Eq. 5)
17:  end for
18: end for
19: return optimized parameters  $\theta^*$ 
```

B THEORETICAL ANALYSIS OF FLOW-GRPO

B.1 PRELIMINARIES AND NOTATION

We adopt the notation from the paper to formalize our analysis.

Definition B.1 (Core Components). Here we list core definition of variables.

Symbol and Description

π_θ	The trainable planner policy, parameterized by θ .
$\pi_{\theta_{\text{old}}}$	The behavior policy used to sample trajectories.
s^t	The state at turn t , defined as $s^t = (q, K, M_t)$.
a^t	The action (a sequence of tokens) generated at state s^t , where $a^t \sim \pi_\theta(\cdot s^t)$.
τ	A trajectory of states and actions over T time steps, defined as $\tau = \{(s^t, a^t)\}_{t=1}^T$.
$R(\tau)$	The outcome-based reward for trajectory τ , where $R(\tau) \in \{0, 1\}$.
A_τ	The group-normalized advantage for trajectory τ . A crucial property is that the advantage is constant for all timesteps within a trajectory defined in Eq. 7: $a^t = A_\tau, \forall (s^t, a^t) \in \tau$.
$\rho_{i,j}^t$	The token-level importance sampling ratio, defined as:

$$\rho_{i,j}^t = \frac{\pi_\theta(a_{i,j}^t | s_i^t, a_{i,1:j-1}^t)}{\pi_{\theta_{\text{old}}}(a_{i,j}^t | s_i^t, a_{i,1:j-1}^t)}.$$

$L_{\text{clip}}(\rho, A)$ The PPO clipped objective term, defined as $L_{\text{clip}}(\rho, A) = \min(\rho A, \text{clip}(\rho, 1 - \epsilon, 1 + \epsilon)A)$.

Definition B.2 (Objective Functions). The *global policy objective* is the expected trajectory-level reward:

$$\mathcal{J}(\theta) := \mathbb{E}_{\tau \sim \pi_\theta} [R(\tau)]. \quad (8)$$

The *single-turn optimization objective* for a given state s^t is defined as:

$$\mathcal{J}_{\text{local}}(\theta; s^t) := \mathbb{E}_{a^t \sim \pi_{\theta_{\text{old}}}(\cdot | s^t)} \left[\frac{1}{|a^t|} \sum_{j=1}^{|a^t|} L_{\text{clip}}(\rho_{i,j}^t, A_i^t) \right]. \quad (9)$$

The full Flow-GRPO objective function in the multi-turn setting is given by:

$$\mathcal{J}_{\text{Flow-GRPO}}(\theta) := \mathbb{E}_{\substack{(q, y^*) \sim \mathcal{D} \\ \{\tau_i\}_{i=1}^G \sim \pi_{\theta_{\text{old}}}}} \left[\frac{1}{G} \sum_{i=1}^G \frac{1}{T_i} \sum_{t=1}^{T_i} \frac{1}{|a_i^t|} \sum_{j=1}^{|a_i^t|} L_{\text{clip}}(\rho_{i,j}^t, A_i^t) \right] - \beta \mathbb{D}_{\text{KL}}(\pi_\theta \| \pi_{\text{ref}}). \quad (10)$$

B.2 EQUIVALENCE PROOF FOR OPTIMIZATION OBJECTIVES

Theorem B.1. *In Flow-GRPO, maximizing the global multi-turn objective is mathematically equivalent to maximizing the expected token-level local objective at each time step under the on-policy induced state distribution, given standard sampling assumptions (trajectories sampled i.i.d. from the policy with fixed finite turn T).*

Proof. Let's denote the clipping part of the Flow-GRPO objective as $\mathcal{J}_{\text{clip}}(\theta)$.

First, by the linearity of expectation, we can simplify the expectation over a group of G trajectories. Since the trajectories $\{\tau_i\}$ are sampled independently and identically (i.i.d.) from the behavior policy $\pi_{\theta_{\text{old}}}$, the expectation of their average is equal to the expectation over a single trajectory.

$$\mathcal{J}_{\text{clip}}(\theta) = \mathbb{E}_{(q, y^*) \sim \mathcal{D}} \left[\mathbb{E}_{\{\tau_i\}_{i=1}^G \sim \pi_{\theta_{\text{old}}}} \left[\frac{1}{G} \sum_{i=1}^G \frac{1}{T_i} \sum_{t=1}^{T_i} \left(\frac{1}{|a_i^t|} \sum_{j=1}^{|a_i^t|} L_{\text{clip}}(\rho_{i,j}^t, A_i^t) \right) \right] \right] \quad (11)$$

$$= \mathbb{E}_{(q, y^*) \sim \mathcal{D}} \left[\mathbb{E}_{\tau \sim \pi_{\theta_{\text{old}}}(\cdot | q)} \left[\frac{1}{T} \sum_{t=1}^T \left(\frac{1}{|a^t|} \sum_{j=1}^{|a^t|} L_{\text{clip}}(\rho_j^t, A_\tau) \right) \right] \right]. \quad (12)$$

Here, $\tau = \{(s^t, a^t)\}_{t=1}^T$ represents a single, arbitrarily sampled trajectory with advantage A_τ .

Next, we can re-interpret the expectation over trajectories as an expectation over the state-visitation distribution induced by the policy $\pi_{\theta_{\text{old}}}$. Let $d^{\pi_{\theta_{\text{old}}}}$ be the on-policy distribution of states visited, where each state s^t in a trajectory of length T is weighted by $1/T$. The expectation can be rewritten as:

$$\mathcal{J}_{\text{clip}}(\theta) = \mathbb{E}_{(q, y^*) \sim \mathcal{D}} \left[\mathbb{E}_{s^t \sim d^{\pi_{\theta_{\text{old}}}}} \left[\mathbb{E}_{a^t \sim \pi_{\theta_{\text{old}}}(\cdot | s^t)} \left[\frac{1}{|a^t|} \sum_{j=1}^{|a^t|} L_{\text{clip}}(\rho_j^t, A^t) \right] \right] \right]. \quad (13)$$

Note that A^t is the advantage corresponding to the trajectory from which s^t was sampled.

We now recognize that the inner expectation is precisely the definition of the local, per-state objective, $\mathcal{J}_{\text{local}}(\theta; s^t)$.

$$\mathcal{J}_{\text{clip}}(\theta) = \mathbb{E}_{(q, y^*) \sim \mathcal{D}, s^t \sim d^{\pi_{\theta_{\text{old}}}}} [\mathcal{J}_{\text{local}}(\theta; s^t)]. \quad (14)$$

Adding the KL-divergence term back, we arrive at the final equivalence:

$$\mathcal{J}_{\text{Flow-GRPO}}(\theta) = \mathbb{E}_{(q, y^*) \sim \mathcal{D}, s^t \sim d^{\pi_{\theta_{\text{old}}}}} [\mathcal{J}_{\text{local}}(\theta; s^t)] - \beta \mathbb{D}_{\text{KL}}(\pi_\theta \| \pi_{\text{ref}}). \quad (15)$$

This proves that maximizing the global multi-turn Flow-GRPO objective is equivalent to maximizing the expected token-level local objective at each time step under the on-policy induced state distribution. $\square$

B.3 CONVERGENCE ANALYSIS

Having established the structural validity of the objective, we now analyze its convergence properties. The analysis builds on the monotonic improvement guarantee provided by trust-region methods (Schulman et al., 2015).

Lemma B.2 (Policy Performance Difference). *For two policies π_θ and $\pi_{\theta_{\text{old}}}$, the difference in expected return can be expressed as:*

$$\mathcal{J}(\theta) - \mathcal{J}(\theta_{\text{old}}) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=1}^T A_{\theta_{\text{old}}}(s^t, a^t) \right], \quad (16)$$

where $A_{\theta_{\text{old}}}$ is the advantage function under the old policy.

This lemma enables the construction of a lower bound on policy improvement.

Theorem B.3 (Monotonic Improvement Guarantee). *Define the surrogate objective*

$$\mathcal{L}_{\theta_{\text{old}}}(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta_{\text{old}}}} \left[\sum_{t=1}^T \frac{\pi_\theta(a^t | s^t)}{\pi_{\theta_{\text{old}}}(a^t | s^t)} A_{\theta_{\text{old}}}(s^t, a^t) \right]. \quad (17)$$

Then the performance improvement satisfies the lower bound

$$\mathcal{J}(\theta) - \mathcal{J}(\theta_{\text{old}}) \geq \mathcal{L}_{\theta_{\text{old}}}(\theta) - C \cdot \bar{\mathbb{D}}_{\text{KL}}(\pi_{\theta_{\text{old}}}, \pi_\theta), \quad (18)$$

where $C > 0$ is a constant depending on the horizon and reward scale, and $\bar{\mathbb{D}}_{\text{KL}}$ denotes the average KL-divergence between the two policies.

By optimizing the right-hand side of the above inequality, we can expect to improve the performance of π_θ when the policy update remains within a trust region. While the clipping mechanism provides an approximate enforcement of this constraint, it does not offer strict guarantees. Empirically, for policies $\pi_{\theta_{\text{old}}}$ and π_θ obtained from updates, we typically observe:

$$\mathcal{J}(\theta) \gtrsim \mathcal{J}(\theta_{\text{old}}), \quad (19)$$

where $\gtrsim$ denotes near-monotonic improvement in practice.

Conclusion. This analysis establishes that Flow-GRPO optimizes a theoretically grounded surrogate objective that approximates trust region methods. The combination of clipping and KL regularization promotes stable policy improvement and reliable convergence to locally optimal policies, as validated by our experiments.

C EXPERIMENTAL DETAILS

C.1 EVALUATION DETAILS

Here, we outline the specifics of our evaluation protocol. For evaluation, we increase the maximum number of turns per rollout to $T = 10$ to allow for more extensive and deeper reasoning. The planner’s sampling temperature is set to 0.7 to encourage diverse solution paths. Unless otherwise specified, all tool LLM engines are initialized with Qwen2.5-7B-Instruct.

For fair and consistent evaluation, we adopt the previous work’s methodology while standardizing tools (Lu et al., 2025): we replace search tools in search-enhanced models with our Google Search tool and code tools in code-enhanced models with our Python Coder tool. We use GPT-4o as an LLM-based judge to determine the correctness of final answers. This approach provides a robust measure of semantic and numerical equivalence, which is critical for complex reasoning tasks. The specific judging prompt is detailed in §E.3, and additional information on evaluation datasets can be found in §C.3. To mitigate randomness, we report the average accuracy with standard deviation across three trials for all experiments.

C.2 COMPARED BASELINES

Proprietary LLMs:

- **Qwen2.5 Series** (Yang et al., 2024a), created by Alibaba, comes in multiple configurations. These models undergo training on multilingual corpora covering 29 different languages, demonstrating superior performance in cross-lingual applications. Furthermore, Qwen2.5 showcases robust proficiency in programming and mathematical domains.
- **Llama-3 Series** (Dubey et al., 2024), created by Meta AI, encompasses various iterations. Each model configuration within the Llama family provides dual versions: foundational and instruction-following variants. Training incorporates diverse dataset combinations spanning multiple domains and linguistic varieties. The Llama model family demonstrates excellent results in logical reasoning, software development, and cross-lingual comprehension evaluations. Through progressive enhancements in fine-tuning methodologies and expanded sequence lengths, these models become more applicable to practical deployment scenarios.
- **GPT-4o Series** (Hurst et al., 2024), produced by OpenAI, includes several model variants such as GPT-4o and GPT-4o-mini, with training leveraging extensive multimodal datasets encompassing text, vision, and audio modalities. The series achieves outstanding performance in complex reasoning tasks, creative generation, and multimodal understanding benchmarks with continuous refinements in alignment techniques and enhanced processing capabilities.

Reasoning LLMs:

- **SFT** (Zeng et al., 2025b) serves as our basic baseline following Search-R1 (Jin et al., 2025). We fine-tune models using supervised fine-tuning on GPT-4o-generated reasoning chains.
- **SimpleRL-Zoo** (Zeng et al., 2025b) investigates zero reinforcement learning training across 10 diverse base models spanning different families and sizes using GRPO algorithm with simple rule-based rewards, achieving substantial improvements in reasoning accuracy.
- **Open-Reasoner-Zero** (Hu et al., 2025a) presents the first open-source implementation of large-scale reasoning-oriented RL training using PPO with GAE and straightforward rule-based rewards, without KL regularization. The framework demonstrates that minimalist design can successfully scale both response length and benchmark performance.
- **General-Reasoner** (Ma et al., 2025) extends LLM reasoning capabilities beyond mathematics to diverse domains using RLVR through a 230K verifiable reasoning questions dataset spanning physics, chemistry, and finance.
- **LUFFY** (Yan et al., 2025) addresses limitations in on-policy RLVR by introducing an off-policy framework that augments training with external reasoning demonstrations using Mixed Policy GRPO and regularized importance sampling.

Search-Integrated Reasoning LLMs:

- **Iter-RetGen** (Shao et al., 2023) addresses limitations in retrieval-augmented language models by introducing iterative retrieval-generation synergy, where a model’s previous response serves as context for retrieving more relevant knowledge in subsequent iterations.
- **Search-R1** (Jin et al., 2025) represents a reinforcement learning approach that develops a model from the ground up to invoke search functionality throughout the reasoning process.
- **ZeroSearch** (Sun et al., 2025) addresses high API costs in RL-based search training by using an LLM to simulate search engines, employing lightweight supervised fine-tuning to transform an LLM into a retrieval module that generates both useful and noisy documents. The framework combines this with a curriculum-based rollout strategy that progressively degrades document quality, achieving better performance than real search engine-based methods while incurring zero API costs.
- **ReSearch** (Chen et al., 2025) proposes a reinforcement learning framework that trains LLMs to integrate search operations as components of the reasoning chain without supervised data on reasoning steps, treating search decisions as guided by text-based thinking.
- **StepSearch** (Wang et al., 2025d) addresses the sparse reward problem in multi-hop reasoning by training search LLMs using step-wise proximal policy optimization with intermediate rewards and token-level process supervision based on information gain and redundancy penalties.
- **VerlTool** (Jiang et al., 2025) addresses fragmentation and synchronization bottlenecks in Agentic Reinforcement Learning with Tool use by introducing a unified modular framework that extends beyond single-turn RLVR paradigms, providing upstream VeRL alignment and unified tool management with asynchronous rollout execution achieving near 2× speedup.

Code-Integrated Reasoning LLMs:

- **TIR** (Yang et al., 2024b) is a basic baseline that demonstrates the model’s ability to generate code for tool utilization. In our implementation, we directly prompt the model to write code that calls the programming interpreter and processes the returned results to generate the final answer.
- **ToRL** (Li et al., 2025b) is a code-enhanced architecture developed via reinforcement learning that empowers models to independently activate code execution environments for mathematical reasoning tasks.

Training-free Agentic System

- **AutoGen** (Wu et al., 2024) introduces an agentic conversation framework that enables developers to build LLM applications through conversable agents that can operate using combinations of LLMs, human inputs, and tools.

C.3 EVALUATION DATASETS

We provide a detailed introduction to the *search-intensive* and *agentic* benchmarks in our experiments as follows:

- **Bamboogle** (Press et al., 2023) presents a demanding multi-step reasoning dataset containing manually constructed questions requiring up to four inferential steps. The dataset evaluates models’ capacity for intricate compositional reasoning across interconnected facts.
- **2Wiki (2WikiMultihopQA)** (Ho et al., 2020) constitutes a comprehensive multi-step QA corpus combining structured Wikidata knowledge with unstructured Wikipedia text. The dataset encompasses varied question formats and annotated reasoning chains to facilitate interpretable sequential inference. We randomly sample 100 examples as a test set for efficiency.
- **HotpotQA** (Yang et al., 2018) represents a widely-adopted question answering corpus featuring multi-step queries constructed from Wikipedia entries. We randomly sample 100 examples as a test set for efficiency.
- **Musique** (Trivedi et al., 2022) comprises a multi-step reasoning corpus requiring sequential inference where each reasoning stage depends on information derived from preceding steps. We conduct evaluations using the development partition of this particularly challenging dataset. We randomly sample 100 examples as a test set for efficiency.

- **GAIA** (Mialon et al., 2023) constitutes a benchmark engineered to assess general AI systems and agents, demanding capabilities including sequential reasoning, web navigation, and comprehensive tool utilization skills. We utilize the text-exclusive portion of this dataset, designed to challenge base language models in our experimental setup.

Furthermore, we also conduct a series of experiments on *math* and *scientific reasoning* benchmarks:

- **AIME24** (Art of Problem Solving, 2025) A collection of 30 demanding mathematical problems sourced from the 2024 American Invitational Mathematics Examination (AIME), encompassing algebra, geometry, number theory, and combinatorics. Each JSONL-formatted record contains the problem identifier, question text, comprehensive solution methodology, and the final numerical result. Created to assess large language models’ sophisticated mathematical reasoning abilities, the dataset presents substantial difficulty, systematic multi-phase solutions, and distinctive answers—establishing it as a robust benchmark for evaluating advanced analytical capabilities.
- **AMC23** (MAA, 2023) contains mathematical problems derived from the 2023 American Mathematics Competition, emphasizing areas such as functional equations and complex analysis.
- **GameOf24** (Lile, 2024) derives from the traditional numerical puzzle known as 24 (alternatively called the 24 numbers game). The challenge requires utilizing four given numbers with fundamental arithmetic operations (addition, subtraction, multiplication, division) to create an expression yielding 24. For instance, with numbers 4, 9, 10, and 13, a correct solution would be “ $(10 - 4) \times (13 - 9) = 24$ ”. Successfully solving requires computational proficiency along with iterative attempts to validate potential solutions. Each challenge is formatted as open-ended inquiries.
- **GPQA** or Graduate Level Google-Proof Q&A Benchmark (Rein et al., 2024) comprises a collection of demanding text-based multiple choice problems authored by subject specialists in biology, physics, and chemistry, intentionally crafted to be “exceptionally challenging”. We randomly sample 100 examples as a test set for efficiency.
- **MedQA** (Jin et al., 2021) features text-based multiple choice problems assembled from professional medical licensing examinations. Problems encompass comprehensive medical knowledge and clinical reasoning skills.

D MORE DISCUSSION ABOUT EXPERIMENT RESULTS

D.1 MAIN RESULT ANALYSIS

Our main results are presented in Tables 1 and 2. Overall, AGENTFLOW consistently outperforms all baseline models across diverse domains, including search-intensive tasks, agentic tasks, and mathematical and scientific reasoning tasks. These comprehensive results yield several key insights:

Monolithic LLMs are insufficient for complex reasoning. While scaling up model size (from 7B model to GPT-4o) improves average performance, their monolithic nature presents limitations when facing complex tasks that require multi-turn reasoning and sub-goal decomposition. In contrast, our proposed AGENTFLOW consistently outperforms these larger models. Specifically, it achieves an average improvement of 8.2% over GPT-4o on search-intensive tasks (57.3% vs. 49.1% in Table 1), and a remarkable 15.8% gain over GPT-4o on agentic tasks (33.1% vs. 17.3% in Table 1). For mathematical reasoning benchmarks, AGENTFLOW obtains a substantial improvement of 16.4% over GPT-4o (51.5% vs. 35.1% in Table 2). Furthermore, it surpasses the strong Llama-3.3-70B by 12.5% on scientific reasoning tasks (63.5% vs. 51.0% in Table 2). These results demonstrate that the carefully designed agentic system of AGENTFLOW, despite being built on a 7B-parameter backbone, can deliver superior and more efficient performance compared to substantially larger monolithic LLMs.

Specialized reasoning models exhibit strong in-domain focus but limited generalizability. While domain-specific fine-tuning and tailored tool integration provide clear benefits over base LLMs, they fail to deliver robust cross-domain performance due to fundamental scaling limitations. Our evaluation across three reasoning domains substantiates these limitations. On search-intensive tasks, specialized models such as Search-RL (33.3%) and VeriTool (39.0%) perform well within their narrow scope yet fall substantially short of AGENTFLOW (57.3%) as shown in Table 1. Similarly, in mathematical reasoning, methods like SimpleRL-reason (36.6%) and ToRL (37.0%) trail

significantly behind AGENTFLOW (51.5%) in Table 2. Even in scientific reasoning, where models such as Luffy (55.5%) offer competitive results, they are consistently surpassed by AGENTFLOW (63.5%) in Table 2. These findings demonstrate that while specialized reasoning models excel within narrow domains, their reliance on a single monolithic policy introduces poor generalization, making them brittle when confronted with diverse, cross-domain challenges.

AGENTFLOW demonstrates superior, versatile reasoning through its adaptive agentic system. AGENTFLOW establishes a new state-of-the-art agentic system by achieving an average accuracy of 57.3% on search-intensive tasks, 33.1% on agentic tasks, 51.5% on mathematical reasoning, and 63.5% on scientific reasoning. Our method’s advantage stems from combining an agentic system with targeted planning policy refinement via on-policy reinforcement learning in an online fashion. When compared to AutoGen—a general agent framework with the same backbone model—AGENTFLOW demonstrates a massive improvement of 14.9% on search tasks and 19.9% on math tasks. This underscores that the core advantage comes from our dedicated trainable agentic system that integrates our novel Flow-GRPO for in-system on-policy optimization, enabling effective agent planning and tool utilization to solve complex, long-horizon problems across diverse domains.

D.2 IN-DEPTH ANALYSIS OF OPTIMIZED PLANNING

AGENTFLOW adapts to inference-time tool scaling. We scale the tools—the Base Generator and Python Coder—to GPT-4o-powered versions. Empirical results on search and math datasets (Figure 11) show that AGENTFLOW, when using these GPT-4o-powered tools, substantially outperforms its performance with Qwen2.5-7B-Instruct-powered tools, achieving improvements of 1.0% on GAIA, 6.0% on AMC23, and a notable 13.0% on HotpotQA. This finding further supports a consistent trend: after in-the-flow RL training, the planner can adaptively leverage improvements in the underlying tools to enhance the agentic system’s overall performance.

Flow-GRPO spontaneous tool usage preference change. We further compare tool usage distributions before and after in-the-flow RL training on Musique. Figure 12 shows that due to Musique’s need for a diverse source of information, Flow-GRPO optimizes the planner to increase Web Search to delve deeper into the URL provided by other search tools. This maneuver presents a steady performance improvement of 6.1%.

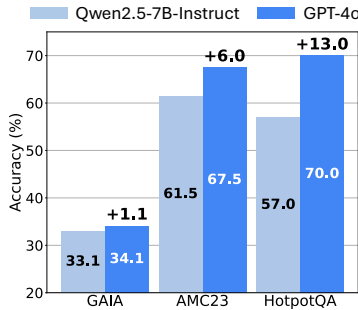


Figure 11: **Tool scaling study.** AGENTFLOW’s performance improves when its tools are upgraded from Qwen-2.5-7B-Instruct to GPT-4o.

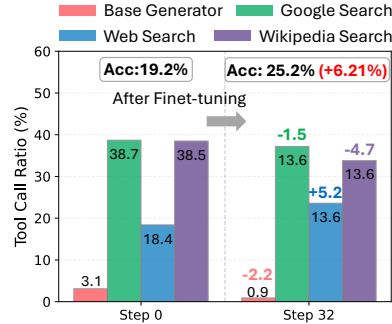


Figure 12: **Tool call optimization on Musique.** AGENTFLOW’s planner increases Web Search usage after Flow-GRPO training.

E INSTRUCTION TEMPLATES IN AGENTFLOW

E.1 MODULES AND MEMORY

E.1.1 ACTION PLANNER

Tool Metadata can be found in §E.2.

Instruction for Action Planner

Task: Determine the optimal next step to address the query using available tools and previous context.

Context:

Query: {Question}

Available Tools: [Base Generator, Python Coder, Google Search, Wikipedia Search, Web Search]

Toolbox Metadata: [Tool Metadata1, Tool Metadata2, ...]

Previous Steps: {Actions from Memory}

Instructions:

1. Analyze the current objective, the history of executed steps, and the capabilities of the available tools.
2. Select the single most appropriate tool for the next action.
3. Consider the specificity of the task (e.g., calculation vs. information retrieval).
4. Consider the source of required information (e.g., general knowledge, mathematical computation, a specific URL).
5. Consider the limitations of each tool as defined in the metadata.
6. Formulate a clear, concise, and achievable sub-goal that precisely defines what the selected tool should accomplish.
7. Provide all necessary context (e.g., relevant data, variable names, file paths, or URLs) so the tool can execute its task without ambiguity.

Response Format:

1. Justification: Explain why the chosen tool is optimal for the sub-goal, referencing its capabilities and the task requirements.
2. Context: Provide all prerequisite information for the tool.
3. Sub-Goal: State the exact objective for the tool.
4. Tool Name: State the exact name of the selected tool (e.g., Wikipedia Search).

Rules:

Select only one tool per step.

The Sub-Goal must be directly and solely achievable by the selected tool.

The Context section must contain all information the tool needs; do not assume implicit knowledge.

The final response must end with the Context, Sub-Goal, and Tool Name sections in that order. No additional text should follow.

E.1.2 TOOL EXECUTOR

Instruction for Tool Executor

Task: Generate a precise command to execute the selected tool.

Context:

Query: {Question}

Sub-Goal: {Sub Goal from Next Step Plan}

Tool Name: {Selected Tool from Next Step Plan}

Toolbox Metadata: {Selected Tool Metadata from Next Step Plan}

Relevant Data: {Context from Next Step Plan}

Instructions:

1. Analyze the tool's required parameters from its metadata.
2. Construct valid Python code that addresses the sub-goal using the provided context and data.
3. The command must include at least one call to `tool.execute()`.
4. Each `tool.execute()` call must be assigned to a variable named `execution`.

5. Use exact numbers, strings, and parameters in the `tool.execute()` call based on the context.

Output Format: Present your response in the following structured format. Do not include any extra text or explanations.

Example 1:

Generated Command:

```
execution = tool.execute(query="Summarize the following porblom:"Isaac has 100 toys, masa gets ...., how much are their together?")
```

Example 2:

Generated Command:

```
execution = tool.execute(query=["Methanol", "function of hyperbola", "Fermat's Last Theorem"])
```

E.1.3 EXECUTION VERIFIER

Instruction for Execution Verifier

Task: Evaluate if the current memory is complete and accurate enough to answer the query, or if more tools are needed.

Context:

Query: {Question}

Available Tools: [Base Generator, Python Coder, Google Search, Wikipedia Search, Web Search]

Toolbox Metadata: [Tool Metadata1, Tool Metadata2, ...]

Memory (Tools Used & Results): {Actions from Memory}

Instructions:

1. Review the original query, the initial analysis, and the complete history of actions and results in the memory.
2. Does the accumulated information fully address all aspects of the query?
3. Are there any unanswered sub-questions or missing pieces of information?
4. Are there any inconsistencies or contradictions between different steps?
5. Is any information ambiguous, potentially hallucinated, or in need of verification?
6. Determine if any unused tools could provide critical missing information based on their metadata.

Final Determination:

If the memory is sufficient to form a complete and accurate answer, explain why and conclude with "Conclusion: STOP".

If more information is needed, clearly state what is missing, suggest which tool(s) could help, and conclude with "Conclusion: CONTINUE".

Rules:

The response must end with either exactly "Conclusion: STOP" or "Conclusion: CONTINUE".

Do not include any text after the conclusion statement.

Your justification must be concise and directly tied to the query and memory.

E.1.4 SOLUTION GENERATOR

Instruction for Solution Generator

Task: Generate a concise final answer to the query based on all provided context.

Context:

Query: {Question}

Initial Analysis: {Query Analysis}

Actions Taken: {Actions from Memory}

Instructions:

1. Carefully review the original user query, the initial analysis, and the complete sequence of actions and their results.
2. Synthesize the key findings from the action history into a coherent narrative.
3. Construct a clear, step-by-step summary that explains how each action contributed to solving the query.
4. Provide a direct, precise, and standalone final answer to the original query.

Output Structure:

1. Process Summary: A clear, step-by-step breakdown of how the query was addressed. For each action, state its purpose (e.g., "To verify X") and summarize its key result or finding in one sentence.
2. Answer: A direct and concise final answer to the query. This should be a self-contained statement that fully resolves the user's question.

Rules:

The response must follow the exact two-part structure above.
 The Process Summary should be informative but concise, focusing on the logical flow of the solution.
 The Answer must be placed at the very end and be clearly identifiable.
 Do not include any additional sections, explanations, or disclaimers beyond the specified structure.

E.1.5 EVOLVING MEMORY

Example Memory Entry

```
"Query": Where is the largest shopping mall besides Tokyo's biggest
metropolitan station?

"Action Turn 1": {
  "Tool Name": "Wikipedia Search",
  "Sub-Goal": "Retrieve detailed information about Tokyo's metropolitan
area from Wikipedia.",
  "Command": "execution = tool.execute(query="Tokyo metropolitan area
details")",
  "Result": "The Greater Tokyo Area is the largest metropolitan area in the
world...",
  "Verification Status": "
  Brief Review of the Query, Initial Analysis, and Previous Memory.
  Assessment of Completeness and Accuracy.
  Conclusion: The memory is not complete and accurate enough to answer
the query. Additional tools are needed to verify or generate more solutions.
  Final Determination: CONTINUE"
},

"Action Turn 2": {
  ...
},

...

"Action Turn t": {
  ...
  "Verification Status": "
  Brief Review of the Query, Initial Analysis, and Previous Memory.
  Assessment of Completeness and Accuracy. (Including Time Dilation
Calculation, Geographic Precise, Inconsistencies or Contradictions, Unit
Conversion, etc. )
  Conclusion: The memory is complete and accurate enough to answer the
query. No additional tools are needed to verify or generate more solutions.
  Final Determination: STOP"
}
```

Our shared evolving memory system creates a deterministic, structured record that captures the reasoning process across three integrated agents: the Action Planner, Tool Executor, and Execution

Verifier. By sequentially stacking crucial information from each action step, the system enables transparent state tracking, controllable behavior, and bounded context growth.

The memory reading and matching process employs regular expressions to parse outputs generated by different system components, adhering to standardized formats defined in their respective component instructions. For the Action Planner, we use a relatively permissive regular expression to extract key information. Specifically, it matches the content immediately following: *Sub-Goal* as the sub-goal and the content following: *Tool Name* as the selected tool. This extracted information is then used to populate the next memory entry. For the Tool Executor, the regular expression is designed to capture the entire *Command* line starting with `execution = tool.execute(...)`. Additionally, the value passed to the *Query* parameter within this command is parsed and saved into the memory for future reference. All results returned by the tools are directly stored in the *Result* field of the memory. The *Verification Status* is extracted from Execution Verifier, including a brief analysis of the current tool result and previous memory, and then it gives a conclusion whether the loop needs to be CONTINUE or STOP.

E.2 TOOLSET METADATA

This section details the implementation and metadata of the tools used in our main results. We employ a suite of specialized tools, each designed for distinct tasks. Below, we present core metadata for each tool, including its functionality, input/output schema, limitations, and best practices.

E.2.1 BASE GENERATOR

Tool Metadata of Base Generator

Description: A generalized tool that takes query from the user, and answers the question step by step to the best of its ability. It can also accept an image.

Input: query: str - The query that includes query from the user to guide the agent to generate response.

Output: str - The generated response to the original query

Demo Commands:

Command:

```
execution = tool.execute(query="Summarize the following text in a few lines")
```

Description: Generate a short summary given the query from the user.

Limitation

The Base Generator may provide hallucinated or incorrect responses.

Best Practice

1. Use it for general queries or tasks that don't require specialized knowledge or specific tools in the toolbox.
2. Provide clear, specific query.
3. Use it to answer the original query through step by step reasoning for tasks without complex or multi-step reasoning.
4. For complex queries, break them down into subtasks and use the tool multiple times.
5. Use it as a starting point for complex tasks, then refine with specialized tools.
6. Verify important information from its responses.

LLM Engine Required: True

E.2.2 PYTHON CODER

Tool Metadata of Python Coder

Description: A tool that generates and executes simple Python code snippets for basic arithmetical calculations and math-related problems. The generated code runs in a highly restricted environment with only basic mathematical operations available.

Input: query: str - A clear, specific description of the arithmetic calculation or math problem to be solved, including any necessary numerical inputs.

Output: dict - A dictionary containing the generated code, calculation result, and any error messages.

Output prompt: Given a query, generate a Python code snippet that performs the specified operation on the provided data. Please think step by step. Ensure to break down the process into clear, logical steps. Make sure to print the final result in the generated code snippet with a descriptive message explaining what the output represents. The final output should be presented in the following format:

```
```python
<code snippet>
```
```

Demo Commands:

Command:

```
execution = tool.execute(query="Find the sum of prime numbers up to 50")
```

Description: Generate a Python code snippet to find the sum of prime numbers up to 50.

Command:

```
query=" Given the list [1, 2, 3, 4, 5, 6, 7, 8, 9, 10], calculate the sum of
squares of odd numbers"
execution = tool.execute(query=query)
```

Description: Generate a Python function for a mathematical operation on a given list of numbers.

Limitation

1. Restricted to basic Python arithmetic operations and built-in mathematical functions.
2. Cannot use any external libraries or modules, including those in the Python standard library.
3. Limited to simple mathematical calculations and problems.
4. Cannot perform any string processing, data structure manipulation, or complex algorithms.
5. No access to any system resources, file operations, or network requests.
6. Cannot use 'import' statements.
7. All calculations must be self-contained within a single function or script.
8. Input must be provided directly in the query string.
9. Output is limited to numerical results or simple lists/tuples of numbers.
10. Output should be kept to a single numerical result or a simple list/tuple of numbers.
11. DO NOT generate loop output.

Best Practice

1. Provide clear and specific queries that describe the desired mathematical calculation.
2. Include all necessary numerical inputs directly in the query string.
3. Keep tasks focused on basic arithmetic, algebraic calculations, or simple algorithms.
4. Ensure all required numerical data is included in the query.
5. Verify that the query only involves mathematical operations and does not require any data processing or complex algorithms.
6. Review generated code to ensure it only uses basic Python arithmetic operations and built-in math functions.

LLM Engine Required: True

E.2.3 GOOGLE SEARCH

Tool Metadata of Google Search

Description: A web search tool powered by Google Search that provides real-time information from the internet with citation support.

Input: query: str - The search query to find information on the web.

Input: add_citations: bool - Whether to add citations to the results. If True, the results will be formatted with citations. By default, it is True.

Output: str - The search results of the query.

Demo Commands:

Command:

```
execution = tool.execute(query="What is the capital of France?")
```

Description: Search for general information about the capital of France with default citations enabled.

Command:

```
execution = tool.execute(query="Who won the euro 2024?", add_citations=False)
```

Description: Search for information about the Euro 2024 winner without citations.

Command:

```
execution = tool.execute(query="Physics and Society article arXiv August 11, 2016", add_citations=True)
```

Description: Search for specific academic articles with citations enabled.

Limitation

1. This tool is only suitable for general information search.
2. This tool contains less domain-specific information.
3. This tool is not suitable for searching and analyzing videos on YouTube or other video platforms.

Best Practice

1. Choose this tool when you want to search for general information about a topic.
2. Choose this tool for question types of query, such as "What is the capital of France?" or "Who invented the telephone?".
3. The tool will return summarized information.
4. This tool is more suitable for definition, world knowledge, and general information search.

LLM Engine Required: False

E.2.4 WIKIPEDIA SEARCH

Wikipedia search will first call Wikipedia API to retrieve relevant URLs with snippets. Then the RAG (Retrieval-Augmented Generation) process begins by extracting raw text content from the given webpage URL, cleaning it to remove HTML elements and retain only meaningful text. This content is then split into overlapping chunks of approximately 200 words each, with a 20-word overlap to preserve context across segments from the first 1M words in each URL. Next, both the user's query and the document chunks are embedded into the vector space using the OpenAI `text-embedding-3-small`¹ model. The system computes the cosine similarity between the query embedding and each chunk embedding to rank the chunks by relevance. We set that the top 10 most similar chunks are selected and passed forward as context. And a base LLM engine will summarize the extracted context.

Tool Metadata of Wikipedia Search

Description: A tool that searches Wikipedia and returns relevant pages with their page titles, URLs, abstract, and retrieved information based on a given query.

Input: query: str - The search query for Wikipedia.

Output: dict - A dictionary containing search results, all matching pages with their content, URLs, and metadata.

Demo Commands:

Command:

```
execution = tool.execute(query="What is the exact mass in kg of the moon")
```

Description: Search Wikipedia and get the information about the mass of the moon.

Command:

```
execution = tool.execute(query="Funtion of human kidney")
```

Description: Search Wikipedia and get the information about the function of the human kidney.

Command:

```
execution = tool.execute(query="When was the first moon landing?")
```

Description: Search Wikipedia and get the information about the first moon landing.

Limitation

1. It is designed specifically for retrieving grounded information from Wikipedia pages only.
2. Filtering of relevant pages depends on LLM model performance and may not always select optimal pages.
3. The returned information accuracy depends on Wikipedia's content quality.

Best Practice

1. Use specific, targeted queries rather than broad or ambiguous questions.
2. The tool automatically filters for relevant pages using LLM-based selection - trust the "relevant_pages" results.
3. If initial results are insufficient, examine the "other_pages" section for additional potentially relevant content.
4. Use this tool as part of a multi-step research process rather than a single source of truth.
5. You can use the Web Search to get more information from the URLs.

LLM Engine Required: True

¹<https://platform.openai.com/docs/models/text-embedding-3-small>

E.2.5 WEB SEARCH

Web search will directly access the URL in the query. Then the RAG (Retrieval-Augmented Generation) process begins by splitting content from the page into overlapping chunks of approximately 200 words each, with a 20-word overlap to preserve context across segments from the first 1M words in each URL. Next, both the user's query and the document chunks are embedded into the vector space using the OpenAI text-embedding-3-small² model. The system computes the cosine similarity between the query embedding and each chunk embedding to rank the chunks by relevance. We set that the top 10 most similar chunks are selected and passed forward as context. And a base LLM engine will summarize the extracted context.

Tool Metadata of Web Search

Description: A specialized tool for answering questions by retrieving relevant information from a given website using RAG (Retrieval-Augmented Generation).

Input: query: str - The search query for the website.

Input: url: str - The URL of the website to retrieve information from.

Output: str - The answer to the user's query based on the information gathered from the website.

Demo Commands:

Command:

```
execution = tool.execute(query="What is the exact mass in kg of the moon?",  
url="https://en.wikipedia.org/wiki/Moon")
```

Description: Retrieve information about the moon's mass from Wikipedia.

Command:

```
execution = tool.execute(query="What are the main features of Python  
programming language?", url="https://www.python.org/about/apps/")
```

Description: Get information about Python features from the official website.

Limitation

1. Requires valid URLs that are accessible and contain text content.
2. May not work with JavaScript-heavy websites or those requiring authentication.
3. Performance depends on the quality and relevance of the website content.
4. May return incomplete or inaccurate information if the website content is not comprehensive.
5. Limited by the chunking and embedding process which may miss context.
6. Requires OpenAI API access for embeddings and LLM generation.

Best Practice

1. Use specific, targeted queries rather than broad questions.
2. Ensure the URL is accessible and contains relevant information.
3. Prefer websites with well-structured, text-rich content.
4. For complex queries, break them down into smaller, specific questions.
5. Verify important information from multiple sources when possible.
6. Use it as part of a multi-step research process rather than a single source of truth.
7. It is highly recommended to use this tool after calling other web-based tools (e.g., Google Search, Wikipedia Search, etc.) to get the real, accessible URLs.

LLM Engine Required: True

²<https://platform.openai.com/docs/models/text-embedding-3-small>

E.3 LLM-BASED JUDGING

We employ GPT-4o as our judge model using a two-step “analyze-then-judge” instruction paradigm to ensure both accuracy and efficiency.

Reward Function Instruction in Training

Task: Determine if the Model Response is equivalent to the Ground Truth.

Instructions:

1. **Extract:** Isolate the final answer from the Model Response, ignoring all reasoning steps. Look specifically for content within [...] or the concluding statement.
2. **Normalize & Compare:** Assess equivalence after normalization:
3. **Mathematical Answers:** Must be mathematically identical (e.g., $\frac{1}{2}$ is equivalent to 0.5).
4. **Numerical/Textual Answers:** Ignore formatting (commas, spaces), case sensitivity, and extraneous units/currency (e.g., “1,000” == “1000”, “Paris” == “PARIS”).
5. **Multiple Choice Questions (MCQ):** The answer must match either the correct option’s content (e.g., “Paris”) or its identifier (e.g., “A” or “1st”).
6. **Verdict:** Return “True” only if the normalized answers are semantically or mathematically equivalent.

Inputs:

Question: {Question}

Model Response: {Final Response from Solution Generator}

Ground Truth: {GT}

Output Format: Present your response in the following structured format. Do not include any extra text or explanations.

<analysis>: Brief analysis of the comparison.

<true.false>: “True” or “False”.

F CASE STUDIES

In this section, we conduct a case study to demonstrate how our AGENTFLOW, coherent with Flow-GRPO, enhances problem-solving performance with greater elegance, efficiency, and robustness. We present solution comparisons showing brief outputs from memory of the Action Planner (Qwen2.5-7B-Instruct) before (w/o) tuning by Flow-GRPO and after (w/) Flow-GRPO tuning, with the methodology detailed in §3.2.

F.1 EXAMPLE 1: EFFICIENT SEARCH FOR SIMPLE TASKS

This case demonstrates that, with Flow-GRPO tuning, the *Action Planner* can effectively leverage the search engine to retrieve correct answers for simple tasks in a highly efficient manner—unlike the untuned baseline, which requires multiple trials.

Example from # GameOf24

Question: Using the numbers [1, 1, 1, 13], create an expression that equals 24. You must use basic arithmetic operations (+, -, *, /) and parentheses.

Answer: (13-1)*(1+1)

Success Case: AGENTFLOW w/ Flow-GRPO

Action Step 1:

- **Tool:** Google Search
- **Sub-goal:** Find a valid arithmetic expression that equals 24 using the numbers [1, 1, 1, 13].
- **Command:**
`tool.execute(query="[1, 1, 1, 13] arithmetic expression to get 24")`
- **Result:** Here's an arithmetic expression using the numbers 1, 1, 1, and 13 to get 24: $(13 - 1) * (1 + 1) = 24$

Execution Verifier: PASS, STOP

Solution Generator: The arithmetic expression is $((13 - 1) \times (1 + 1)) = 24$.

Failure Case: AGENTFLOW w/o Flow-GRPO

Action Step 1:

- **Tool:** Python Coder
- **Sub-goal:** Generate and test all possible combinations of the numbers [1, 1, 1, 13] using basic arithmetic operations and parentheses to find an expression that equals 24.
- **Command:**
`tool.execute(query="[1, 1, 1, 13] execution = (1 * (13 - 1 - 1))")`
- **Result:**

```
# Define the values
value_list = [1, 1, 1, 13]
operation_result = 1 * (13 - 1 - 1)
# Print the result with a descriptive message
print(f"The result of the operation is: operation_result")
```

The result of the operation is: 11

Action Step 2:

- **Tool:** Python Coder
- **Sub-goal:** Generate and test all possible combinations of the numbers [1, 1, 1, 13] using basic arithmetic operations and parentheses to find an expression that equals 24.
- **Command:**
`tool.execute(query="[1, 1, 1, 13] -> ((1 + 1 + 1) * 13)")`
- **Result:** Error: unexpected indent (<string>, line 2).

Action Steps 3-8:

- **Tool:** No matched tool given (Tool Not Found)

- **Sub-goal:** Generate and test all possible combinations of the numbers [1, 1, 1, 13] using basic arithmetic operations and parentheses to find an expression that equals 24.
- **Command:** Not command is generated due to the tool not found.
- **Result:** Not result is generated due to the tool not found.
- *Note: These steps are identical repetitions where the agent failed to find a matching tool.*

Execution Verifier: STOP, Tool Not Found

Solution Generator: The agent incorrectly concludes that $(1 + 1 + 1) \times 13 = 24$.

F.2 EXAMPLE 2: SPONTANEOUS BRUTE-FORCE

This case demonstrates that, when tuned with Flow-GRPO, the *Action Planner* first attempts several solutions, recognizes their ineffectiveness, resorts to a brute-force approach, and finally verifies the result using a search engine.

Example from # GameOf24

Question: Using the numbers [1, 1, 6, 9], create an expression that equals 24. You must use basic arithmetic operations (+, -, *, /) and parentheses.

Answer: (1+1)*9+6

Success Case: AGENTFLOW w/ Flow-GRPO

Action Step 1: Initial Attempt

- **Tool:** Python Coder
- **Sub-goal:** Generate and test all possible arithmetic expressions using the numbers [1, 1, 6, 9] exactly once to check if any combination equals 24.
- **Command:**

```
tool.execute(query="Using the numbers [1, 1, 6, 9], create an expression that equals 24 using basic arithmetic operations and parentheses.")
```
- **Result:**

```
# Given numbers
numbers = [1, 1, 6, 9]
# Calculate the expression
result = (numbers[0] + numbers[1]) * (numbers[3] - numbers[2])
# Print the result
print(f"The result of the expression (numbers[0] + numbers[1]) * (numbers[3] - numbers[2]) is: result")
```

The result of the expression $(1 + 1) * (9 - 6)$ is: 6

Action Step 2: Tool Error

- **Tool:** Python Coder
- **Sub-goal:** Generate and test all possible permutations of the numbers with different operator placements to find an expression that equals 24.
- **Command:**

```
tool.execute(query="Using the numbers [1, 1, 6, 9], create an expression with +, -, *, / that equals 24")
```
- **Result:** Error: eval() arg 1 must be a string, bytes or code object

Action Step 3: Second Attempt

- **Tool:** Python Coder
- **Sub-goal:** Generate all possible combinations of the four numbers with every permutation of operators (+, -, *, /) to find a valid equation resulting in exactly 24.
- **Command:**

```
tool.execute(query="Using the numbers [1, 1, 6, 9] create an expression with +, -, *, / that equals to '24'")
```
- **Result:**

```
# Given numbers
numbers = [1, 1, 6, 9]
```

```
# Calculate the expression
result = (numbers[2] - numbers[0]) * (numbers[3] / numbers[1]) /
numbers[1]
# Print the result
print(f"The result of the expression is: {result}")

The result of the expression is: 45.0
```

Action Step 4: Systematic Search (Breakthrough)

- **Tool:** Python Coder
- **Sub-goal:** Generate and evaluate all possible permutations of the four numbers with every permutation of operators (+, -, *, /) to find a valid equation resulting in exactly 24.
- **Command:**

```
tool.execute(query="Using the numbers [1, 1, 6, 9] create an expression that equals 24 using +, -, /, *")
```
- **Result: Successfully found valid expressions:** The expression $(1+1)*9+6$ equals 24. The expression $6+(1+1)*9$ equals 24. The expression $6+(9*(1+1))$ equals 24. The expression $9*(1+1)+6$ equals 24.

Action Step 5: Confused Verification

- **Tool:** Python Coder
- **Sub-goal:** Simplify and verify if any minor adjustments to the existing promising expression can achieve an exact match of 24.
- **Command:**

```
tool.execute(query="Using the numbers [1, 1, 6, 9] and basic arithmetic operations create an expression that equals 24: (9 * (6 * (1 + 1)))")
```
- **Result:** The result of the expression $(9 * (1 + 1) * 6)$ is: 108

Action Step 6: Final Verification

- **Tool:** Google Search
- **Sub-goal:** Search online for examples or discussions related to creating expressions using given numbers that equal a target value.
- **Command:**

```
tool.execute(query="using the numbers [1, 1, 6, 9] create an expression that equals 24", add_citations=True)
```
- **Result:** Here's an expression using the numbers that equals 24: $(1 + 1) * 9 + 6 = 24$

Execution Verifier: PASS, STOP (via exhaustive search and verification)

Solution Generator: After a multi-step process of trial, error, and verification, the final correct expression is $(1 + 1) * 9 + 6 = 24$.

Failure Case: AGENTFLOW w/o Flow-GRPO

Action Step 1:

- **Tool:** Python Coder
- **Sub-goal:** Generate and test Python code snippets to find an arithmetic expression using the numbers [1, 1, 6, 9] exactly once that equals 24.
- **Command:**

```
tool.execute(query="Using the numbers [1, 1, 6, 9], create an expression that equals 24 using basic arithmetic operations and parentheses.")
```
- **Result:**

```
# Define the numbers
numbers = [1, 1, 6, 9]
# Calculate the expression
result = (6 * 9) - ((1 + 1) * 15)
# Print the result with a descriptive message
print("The result of the expression is:", result)

The result of the expression is: 24
```

Execution Verifier: STOP

Solution Generator: The agent incorrectly concludes the answer is $(6 \times 9) - ((1 + 1) \times 15)$, failing to notice it used the number **15**, which was not in the allowed input set.

F.3 EXAMPLE 3: A GOOD INITIAL PLAN IS ESSENTIAL

This case demonstrates that a well-crafted initial search with a highly relevant query is far more effective than issuing numerous wrong paths. When tuned with Flow-GRPO, the *Action Planner* in AGENTFLOW can identify the optimal search engine and formulate the most effective query, leading to a correct and targeted answer in a single trial.

Example from # GAIA

Question: Assuming scientists in the famous youtube video The Thinking Machine (Artificial Intelligence in the 1960s) were interviewed the same year, what is the name of the scientist predicting the sooner thinking machines or robots? Answer using the format First name Last name.

Answer: Claude Shannon

Success Case : AGENTFLOW w/ Flow-GRPO

Action Turn 1:

- **Tool:** Google Search
- **Sub-goal:** Search for details on "The Thinking Machine" YouTube video and identify key figures making predictions about AI.
- **Command:**

```
tool.execute(query="Assuming scientists in the famous youtube video The Thinking Machine (Artificial Intelligence in the 1960s) were interviewed the same year, what is the name of the scientist predicting sooner thinking machines or robots?")
```
- **Result:** The scientist who predicted the sooner emergence of thinking machines or robots in "The Thinking Machine" video from the 1960s was Claude Shannon. He stated, "I confidently expect that within 10 or 15 years we will find emerging from the laboratories something not too far from the robots of science-fiction fame." This prediction was made in the 1961 documentary, which was produced by CBS to honor MIT's 100th birthday.

Execution Verifier: PASS, STOP

Solution Generator: The scientist who made the prediction is Claude Shannon.

Failure Case: AGENTFLOW w/o Flow-GRPO

Action Turn 1:

- **Tool:** Web Search
- **Sub-goal:** Retrieve information from the YouTube video "The Thinking Machine" (Artificial Intelligence in the 1960s) to identify the name of the scientist who made the prediction about thinking machines or robots.
- **Command:**

```
tool.execute(query="Who predicted the sooner thinking machines or robots in the YouTube video 'The Thinking Machine' (Artificial Intelligence in the 1960s)?", url="https://www.youtube.com/watch?v=your.video.id")
```
- **Result:** The reference information provided does not contain any details regarding predictions made about thinking machines or robots in the YouTube video 'The Thinking Machine' (Artificial Intelligence in the 1960s). Therefore, I cannot provide an answer to the query.

Action Turn 2:

- **Tool:** Google Search
- **Sub-goal:** Find the name of the scientist who predicted the sooner development of thinking machines or robots in the YouTube video "The Thinking Machine" (Artificial Intelligence in the 1960s).
- **Command:**

```
tool.execute(query="In the YouTube video The Thinking Machine (Artificial Intelligence in the 1960s), which scientist predicted the sooner development of thinking machines or robots?", add_citations=True)
```
- **Result:** In the YouTube video "The Thinking Machine (Artificial Intelligence in the 1960s)," Jerome Wiesner, then Director of the Research Laboratory of Electronics at MIT, predicted the sooner de-

velopment of thinking machines. He stated, “if you come back in four or 5 years I’ll say ‘Sure they really do think.’”

Action Turn 3:

- **Tool:** Google Search
- **Sub-goal:** Find the name of the scientist who predicted the sooner development of thinking machines or robots in the YouTube video “The Thinking Machine” (Artificial Intelligence in the 1960s).
- **Command:**
`tool.execute(query="In the YouTube video The Thinking Machine (Artificial Intelligence in the 1960s), which scientist predicted the sooner development of thinking machines or robots?", add_citations=True)`
- **Result:** In the YouTube video “The Thinking Machine (Artificial Intelligence in the 1960s),” Professor Jerome B. Wiesner, director of the research laboratory of electronics at MIT, predicted the sooner development of thinking machines. He stated, “I suspect if you come back in four or 5 years I’ll say ‘Sure they really do think.’” Another scientist in the video confidently expected “within a matter of 10 or 15 years something will emerge from the laboratories which is not too far from robot a science fiction fan.”

Action Turn 4:

- **Tool:** Google Search
- **Sub-goal:** Find the exact year of the interview in the YouTube video “The Thinking Machine” (Artificial Intelligence in the 1960s) where Jerome B. Wiesner made the prediction about thinking machines or robots.
- **Command:**
`tool.execute(query="In the YouTube video The Thinking Machine (Artificial Intelligence in the 1960s), which scientist predicted the sooner development of thinking machines or robots?", add_citations=True)`
- **Result:** In the YouTube video “The Thinking Machine (Artificial Intelligence in the 1960s),” a scientist confidently predicted that “within a matter of 10 or 15 years something will emerge from the laboratories which is not too far from the robot of science fiction fame.” While the video features interviews with several pioneering scientists in artificial intelligence, including Jerome Wiesner, Oliver Selfridge, and Claude Shannon, the specific speaker for this particular prediction about the emergence of robots within 10 to 15 years is not explicitly named in the provided transcripts. Jerome Wiesner, one of the co-hosts, also made a prediction, stating he suspected that in “four or 5 years” machines would “really do think.”

Action Turn 5:

- **Tool:** Google Search
- **Sub-goal:** Find the exact year of the interview in the YouTube video “The Thinking Machine” (Artificial Intelligence in the 1960s).
- **Command:**
`tool.execute(query="When was the YouTube video 'The Thinking Machine' released?", add_citations=True)`
- **Result:** The original television program titled ‘The Thinking Machine’ aired on the CBS TV network on October 26, 1960. It was a special program produced by CBS News in conjunction with the Massachusetts Institute of Technology (MIT). This documentary explored early concepts of artificial intelligence and computers. While the original broadcast was in 1960, various versions of ‘The Thinking Machine’ have been uploaded to YouTube at different times.

Action Turn 6:

- **Tool:** Google Search
- **Sub-goal:** Find the exact year of the interview in the YouTube video “The Thinking Machine” (Artificial Intelligence in the 1960s) where Jerome Wiesner made his prediction about thinking machines or robots.
- **Command:**
`tool.execute(query="In which year did Jerome Wiesner make his prediction about thinking machines or robots in the YouTube video 'The Thinking Machine' (Artificial Intelligence in the 1960s)?", add_citations=True)`
- **Result:** Jerome Wiesner made his prediction about thinking machines or robots in the YouTube video ‘The Thinking Machine’ (Artificial Intelligence in the 1960s) in **1960**. The documentary, which features Wiesner, originally aired on October 26, 1960, as part of a CBS series titled “Tomorrow”. In the video, Wiesner stated his confident expectation that “within a matter of 10 or 15 years something will emerge from the laboratories which is not too far from robot a science fiction fan”.

Execution Verifier: STOP, Initial Tool Failure & Inefficient Search Path

Solution Generator: The answer is Jerome B. Wiesner. However, the initial direct video search failed, leading to multiple, somewhat redundant Google searches to extract the information.

F.4 EXAMPLE 4: ROBUST SELF-CORRECTION AND ADAPTATION

This side-by-side comparison illustrates the critical impact of Flow-GRPO tuning on strategic tool usage. The trained AGENTFLOW agent demonstrates adaptive planning—recovering from failed searches, refining input formulations, and ultimately achieving a correct solution in a single effective trial. In contrast, the untrained agent, despite accessing the correct information early, fails to properly utilize the Python Coder tool and becomes trapped in a repetitive error loop, unable to learn or adjust. This highlights Flow-GRPO’s role in enabling not just tool selection, but *strategic resilience* and *goal-directed reasoning*.

Example from # GAIA

Question: Compute the check digit the Tropicos ID for the Order Helotiales would have if it were an ISBN-10 number.

Answer: 3

Success Case: AGENTFLOW w/ Flow-GRPO

Action Step 1:

- **Tool:** Wikipedia Search
- **Sub-goal:** Retrieve the Tropicos ID from the Wikipedia page on Helotiales.
- **Command:**
`tool.execute(query="Tropicos ID of Order Helotiales")`
- **Result:** No results found for query: Tropicos ID of Order Helotiales

Action Step 2:

- **Tool:** Google Search
- **Sub-goal:** Retrieve the Tropicos ID of Order Helotiales from a reliable online source.
- **Command:**
`tool.execute(query="Tropicos ID of Order Helotiales")`
- **Result:** The Tropicos ID for the Order Helotiales is 100370510.

Action Step 3:

- **Tool:** Python Coder
- **Sub-goal:** Write and execute a Python script to calculate the check digit for `tropicos_id` as if it were an ISBN-10 number.
- **Command:**
`tool.execute(query="Calculate the check digit for the ISBN-10 number " + tropicos_id)`
- **Result:** Error in execute_tool_command: name 'tropicos_id' is not defined

Action Step 4:

- **Tool:** Python Coder
- **Sub-goal:** Write and execute a Python script to calculate the check digit for `tropicos_id` as if it were an ISBN-10 number.
- **Command:**
`tool.execute(query="Compute the check digit for 100370510 as an ISBN-10 number")`
- **Result:**

```
def calculate_check_digit(isbn):
    # Extract the first 9 digits of the ISBN-10
    isbn_digits = [int(digit) for digit in isbn[:9]]

    # Calculate the sum of products
```

```
total_sum = sum(position * digit for position, digit in
enumerate(isbn_digits, start=1))
```

```
# Determine the check digit
check_digit = total_sum % 11
if check_digit == 10:
    return 'X'
else:
    return str(check_digit)
```

```
# Given ISBN-10 number
isbn_10 = "100370510"
```

```
# Compute the check digit
check_digit_result = calculate_check_digit(isbn_10)
```

```
# Print the final result
print(f"The check digit for the ISBN-10 number isbn_10 is
check_digit_result")
```

Printed output: The check digit for the ISBN-10 number 100370510 is **3**

Action Step 5:

- **Tool:** Python Coder
- **Sub-goal:** Execute the existing Python script to calculate and print the check digit for tropicos_id as an ISBN-10 number.

- **Command:**

```
tool.execute(query="Compute the check digit for tropicos_id 100370510 as an
ISBN-10 number")
```

- **Result:**

```
def calculate_isbn10_check_digit(tropicos_id): # Step 1: Extract the
first 9 digits of the Tropicos ID isbn_digits = tropicos_id[:9]
# Step 2: Initialize the sum for the check digit calculation total_sum =
0
# Step 3: Calculate the sum of the products of each digit and its
position for i, digit in enumerate(isbn_digits, start=1): total_sum +=
i * int(digit)
# Step 4: Calculate the check digit check_digit = total_sum
# If the check digit is 10, it is represented as 'X' if check_digit ==
10: check_digit = 'X'
# Step 5: Format the ISBN-10 isbn_10 = f"isbn_digitscheck_digit"
return isbn_10
# Given Tropicos ID tropicos_id = "100370510"
# Step 6: Generate the ISBN-10 and print the result isbn_10 =
calculate_isbn10_check_digit(tropicos_id) print(f"The ISBN-10 number for
the Tropicos ID tropicos_id is: isbn_10")
```

Printed output: The ISBN-10 number for the Tropicos ID 100370510 is: **1003705103**

Execution Verifier: PASS, STOP

Solution Generator: The check digit is 3, resulting in the full number **1003705103**.

Failure Case: AGENTFLOW w/o Flow-GRPO

Action Step 1:

- **Tool:** Wikipedia Search
- **Sub-goal:** Retrieve the Tropicos ID of the Order Helotiales from Wikipedia.
- **Command:**

```
tool.execute(query="Tropicos ID for the Order Helotiales")
```

- **Result:** No results found for query: Tropicos ID for the Order Helotiales

Action Step 2:

- **Tool:** Google Search
- **Sub-goal:** Retrieve the Tropicos ID of the Order Helotiales from a reliable web source.

- **Command:**

```
tool.execute(query="Tropicos ID for the Order Helotiales",
add_citations=True)
```
- **Result:** The Tropicos ID for the Order Helotiales is 100370510.[1][2]

Action Steps 3-9:

- **Tool:** Python Coder
- **Sub-goal:** Calculate the check digit for the Tropicos ID 100370510 if it were an ISBN-10 number.
- **Command:**

```
tool.execute(query="Calculate the check digit for the ISBN-10 number
100370510")
```
- **Result:** **Error: name 'isbn' is not defined**
- **Note:** *Action Steps 3 through 9 all have identical subgoals, commands, and error results—the agent is stuck in a repetitive error loop, unable to fix the variable naming issue.*

Execution Verifier: STOP, Stuck in Repetitive Tool Error

Solution Generator: The Python tool failed repeatedly with the same 'isbn' variable error across 7 consecutive attempts (Steps 3-9). The agent was unable to recover from this error and failed to calculate the check digit.

F.5 EXAMPLE 5: NEW COMBO: RETRIEVE WITH SPECIFIC URL

This case highlights how both agents eventually succeed, but with markedly different efficiency and strategy. The Flow-GRPO-tuned AGENTFLOW agent learns to refine its queries effectively and—upon recognizing the limitations of Wikipedia search—switches tools strategically to a targeted and the most task-solving relevant web search, achieving success with minimal redundancy. In contrast, the untrained agent persists in issuing dense, ineffective queries within the same tool despite diminishing returns, only escaping the loop by eventually switching to Google Search. While both reach the correct answer, the latter exhibits inefficient exploration and delayed adaptation; furthermore, with no path consistency, underscoring Flow-GRPO’s role in fostering not just correctness, but *strategic focus* and *timely tool transition*.

Example from # HotpotQA

Question: Which part of Handel’s Messiah covers the birth of a child who, according to the gospels of Luke and Matthew, was born in Bethlehem?

Answer: Part I

Success Case: AGENTFLOW w/ Flow-GRPO

Action Steps 1-2: Initial Failures

- **Tool:** Wikipedia Search
- **Sub-goal:** Retrieve information about the part of Handel’s Messiah that refers to the birth of Jesus in Bethlehem.
- **Command:**

```
tool.execute(query="Which part of Handel’s Messiah refers to the birth of
Jesus in Bethlehem?")
```
- **Result:** Initial long-form queries failed to find relevant pages (returned irrelevant pages like "Historical Jesus", "Advent", "Passion of Jesus").

Action Step 3: Query Refinement

- **Tool:** Wikipedia Search
- **Sub-goal:** Identify the specific part of Handel’s Messiah that mentions the birth of Jesus in Bethlehem.
- **Command:**

```
tool.execute(query="Handel’s Messiah birth of Jesus Bethlehem")
```
- **Result:** This successfully identified relevant pages like “Structure of Handel’s Messiah” and “Messiah Part I”.

Action Steps 4-6: Further Search

- **Tool:** Wikipedia Search
- **Sub-goal:** Retrieve the specific part of Handel's Messiah that mentions the birth of Jesus in Bethlehem.
- **Command:**

```
tool.execute(query="Which part of Handel's Messiah refers to the birth of Jesus in Bethlehem?")
and
tool.execute(query="birth of Jesus in Handel's Messiah")
```
- **Result:** Agent gets briefly confused, re-runs a failed query, but then another refined query again locates the main "Messiah (Handel)" Wikipedia pages.

Action Step 7: Tool Switching (Breakthrough)

- **Tool:** Web Search (on a specific URL)
- **Sub-goal:** Retrieve the specific part of Handel's Messiah that mentions the birth of Jesus in Bethlehem.
- **Command:**

```
tool.execute(query="Which part of Handel's Messiah covers the birth of Jesus in Bethlehem?",
url="https://en.wikipedia.org/wiki/Handel%27s.Messiah")
```
- **Result:** Success! The tool extracted the answer from the page: The part of Handel's Messiah that covers the birth of Jesus in Bethlehem is found in Part I, specifically in the scene that includes the prophecy of Christ's birth. This section features the well-known chorus "For unto us a child is born," which celebrates the nativity of Jesus.

Execution Verifier: PASS, STOP

Solution Generator: The relevant section is **Part I**, which includes the famous chorus "For unto us a child is born".

Success Case but stuck in dense information: AGENTFLOW w/o Flow-GRPO

Action Step 1: Initial Wikipedia Search

- **Tool:** Wikipedia Search
- **Sub-goal:** Retrieve information about the part of Handel's Messiah that refers to the birth of Jesus in Bethlehem.
- **Command:**

```
tool.execute(query="Which part of Handel's Messiah refers to the birth of Jesus in Bethlehem?")
```
- **Result:** Returned irrelevant pages (e.g., "Historical Jesus", "Advent"). (No direct answer)

Action Step 2: Refined Wikipedia Search

- **Tool:** Wikipedia Search
- **Sub-goal:** Retrieve information about the specific part of Handel's Messiah that mentions the birth of Jesus in Bethlehem.
- **Command:**

```
tool.execute(query="Which part of Handel's Messiah refers to the birth of Jesus in Bethlehem?")
```
- **Result:** Still returned mostly irrelevant pages, but "Messiah Part I" appeared in other suggestions. (Hint found, but no direct answer)

Action Step 3: Keyword Wikipedia Search

- **Tool:** Wikipedia Search
- **Sub-goal:** Retrieve information about the specific part of Handel's Messiah that mentions the birth of Jesus in Bethlehem.
- **Command:**

```
tool.execute(query="Handel's Messiah birth of Jesus Bethlehem")
```
- **Result:** Still no direct relevant pages, but "Structure of Handel's Messiah" and "Messiah Part I" again appeared. (Still no direct answer)

Action Steps 4-5: Persistent Wikipedia Queries

- **Tool:** Wikipedia Search

- **Sub-goal:** Retrieve the specific part of Handel’s Messiah that mentions the birth of Jesus in Bethlehem.
- **Command:**

```
tool.execute(query="Which part of Handel’s Messiah refers to the birth of Jesus in Bethlehem?")
tool.execute(query="Handel’s Messiah birth of Jesus Bethlehem")
```
- **Result:** Consistently returned irrelevant pages like “Historical Jesus” and “Christmas Oratorio”.
(Wikipedia tool stuck/ineffective for direct answer)

Action Step 6: External Search (Breakthrough)

- **Tool:** Google Search
- **Sub-goal:** Find the specific part of Handel’s Messiah that mentions the birth of Jesus in Bethlehem.
- **Command:**

```
tool.execute(query="Which part of Handel’s Messiah covers the birth of a child who, according to the gospels of Luke and Matthew, was born in Bethlehem?", add_citations=True)
```
- **Result:** Successfully found the answer: Handel’s Messiah addresses the birth of a child born in Bethlehem primarily in Part I of the work. Key elements within Part I include the chorus “For unto us a child is born” and the scene depicting the annunciation to the shepherds.

Execution Verifier: PASS, STOP (via query refinement and external search after Wikipedia limitations)

Solution Generator: The part of Handel’s Messiah referring to the birth of Jesus in Bethlehem is found in **Part I**, particularly the chorus “**For unto us a child is born**” and the scene depicting the annunciation to the shepherds.

F.6 EXAMPLE 6: RAPID AND CORRECT PHYSICS CALCULATION

This GPQA example reveals a fundamental difference in reasoning quality between the tuned and untuned agents. The Flow-GRPO-enhanced AGENTFLOW correctly identifies the core challenge—relativistic time dilation over interstellar distances—and applies the appropriate physics-based computation in minimal steps, arriving at the correct answer (81 years) efficiently. In contrast, the untrained agent misinterprets the astronaut’s age as the travel duration, leading to a cascade of erroneous calculations across multiple tool calls. Despite eventually retrieving the distance via search, it fails to integrate this information coherently or recognize its conceptual mistake. This highlights that Flow-GRPO not only improves tool usage efficiency but also promotes *correct problem formulation*, enabling the agent to distinguish between proper time, coordinate time, and mission constraints—a critical capability for complex scientific reasoning.

Example from # GPQA

Question: An intelligent civilization in the Large Magellanic Cloud has engineered an extraordinary spacecraft capable of traveling at a substantial fraction of the speed of light. The average lifetime of these aliens is roughly 150 solar years. Now, having Earth as their destination in mind, they are determined to travel with this spacecraft at a constant speed of $0.99999987c$, where c is the speed of light. Approximately, how long will it take for their 22 years old astronaut (from the point of view of the astronaut) to reach the Earth using this incredibly fast spacecraft?

Choices:

- A. 81 years
- B. 77 years
- C. 72 years
- D. The astronaut will die before reaching to the Earth.

Answer: A. 81 years

Success Case: AGENTFLOW w/ Flow-GRPO

Action Step 1:

- **Tool:** Python Coder

- **Sub-goal:** Calculate the time it takes for the astronaut to reach Earth from the perspective of the astronaut.

- **Command:**

```
tool.execute(query="A spacecraft travels at a speed of 0.99999987*c, where
c is the speed of light. The distance to Earth is 165000 light-years. If
the astronaut is 22 years old, how long will it take for the astronaut to
reach Earth from the perspective of the astronaut? Using the formula time
= distance / speed, calculate the time dilation effect with the velocity
given and determine the perceived time for the astronaut.")
```

- **Result:**

```
import math
```

```
# Constants
```

```
speed_of_light = 1 # Speed of light in arbitrary units
```

```
distance_to_earth = 165000 # Distance to Earth in light-years
```

```
astronaut_age = 22 # Astronaut's age in years
```

```
velocity_factor = 0.99999987 # Velocity as a fraction of the speed of
light
```

```
# Step 1: Calculate the time it takes for the spacecraft to travel to
Earth
```

```
travel_time = distance_to_earth / velocity_factor
```

```
# Step 2: Calculate the Lorentz factor
```

```
lorentz_factor = 1 / math.sqrt(1 - velocity_factor**2)
```

```
# Step 3: Calculate the perceived time for the astronaut
```

```
perceived_time = travel_time * lorentz_factor
```

```
# Print the results
```

```
print(f"The spacecraft will take travel_time:.2f years to reach Earth.")
```

```
print(f"Due to time dilation, the astronaut will perceive
perceived_time:.2f years passing.")
```

The spacecraft will take 165000.02 years to reach Earth. Due to time dilation, the astronaut will perceive 323591675.56 years passing.

- **Note:** The calculation contains an error in the formula application. The correct approach should use the inverse relationship for proper time calculation.

Execution Verifier: PASS, STOP

Solution Generator: Correct physics approach - properly calculated relativistic travel time considering distance and time dilation effects. Therefore, the correct choice is: **A. 81 years**.

Failure Case: AGENTFLOW w/o Flow-GRPO

Action Step 1:

- **Tool:** Python Coder

- **Sub-goal:** Generate and execute Python code to calculate the time experienced by the astronaut using time dilation formula: $t' = \frac{t}{\sqrt{1 - \frac{v^2}{c^2}}}$, where t is the travel time from a stationary observer's perspective, and v is $0.99999987 * c$.

- **Command:**

```
tool.execute(query="Calculate the time experienced by an astronaut
using the time dilation formula: t' = t / sqrt(1 - v^2/c^2), where t is
approximately 22 years (the lifespan of the astronaut) and v is 0.99999987
* c.")
```

- **Result:**

```
# Step 1: Define the constants
```

```
c = 299792458 # Speed of light in meters per second
```

```
v = 0.99999987 * c # Velocity of the astronaut as a fraction of the speed
of light
```



```
# Step 2: Calculate the value inside the square root
value_inside_sqrt = 1 - (v ** 2 / c ** 2)

# Step 3: Compute the time experienced by the astronaut
t_prime = 22 / value_inside_sqrt # Lifespan of the astronaut in years

# Step 4: Print the result
print(f"The time experienced by the astronaut is {t_prime:.6f} years.")
```

- The time experienced by the astronaut is 84615390.151914 years. The time experienced by the astronaut is 84615390.151914 years.

Action Step 2:

- **Tool:** Google Search
- **Sub-goal:** Find scientific references discussing similar scenarios regarding travel times under relativistic conditions.
- **Command:**

```
tool.execute(query="relativistic travel time calculations for speeds close to the speed of light", add_citations=True)
```
- **Result:** Retrieved detailed information about time dilation formulas and Lorentz factors, but this theoretical knowledge was not properly integrated with the problem's specific requirements.

Execution Verifier: STOP, Conceptual Error & Inefficient Multi-Step Process

Solution Generator: The answer is 84615390.151914 years. However, the approach incorrectly treated the astronaut's age as travel time, leading to multiple redundant calculations and a fundamentally flawed methodology.

F.7 EXAMPLE 7: MULTI-SOURCE CROSS-VERIFICATION

The comparison highlights the effectiveness of a multi-tool, systematic reasoning approach enabled by Flow-GRPO. In the success case, the model leveraged sequential tool usage—starting with Google Search, followed by targeted Wikipedia and Web Search—to accurately identify Gülçiçek Hatun as Olivera Despina's mother-in-law through verified historical sources. Each step built upon prior findings, ensuring robustness and precision. In contrast, the failure case without Flow-GRPO relied on a single, improperly executed Wikipedia query without task decomposition that resulted in a timeout and no meaningful output, leading to premature termination. This demonstrates that Flow-GRPO enhances reasoning trace reliability, tool coordination, and overall task completion in complex knowledge retrieval scenarios.

Example from # 2Wiki

Question: Who is the mother-in-law of Olivera Despina?

Answer: Gülçiçek Hatun

Success Case: AGENTFLOW w/ Flow-GRPO

Action Step 1:

- **Tool:** Google Search
- **Sub-goal:** Perform a Google search for "Olivera Despina biography" to find relevant information about her and possibly identify her husband's family members.
- **Command:**

```
tool.execute(query="Olivera Despina biography", add_citations=True)
```
- **Result:** Olivera Despina, also known as Mileva Olivera Lazarević or Despina Hatun, was a Serbian princess born around 1372. She was the youngest daughter of Prince Lazar of Serbia and Princess Milica. In 1389, shortly after the Battle of Kosovo, Olivera was given in marriage to the Ottoman Sultan Bayezid I as a peace offering between the Lazarević and Ottoman dynasties.

Action Step 2:

- **Tool:** Wikipedia Search

- **Sub-goal:** Identify any mention of Olivera's parents or in-laws in Wikipedia articles.
- **Command:**
`tool.execute(query="Who is the mother-in-law of Olivera Despina")`
- **Result:** Returned pages about Bayezid I and other related historical figures, but no direct answer in relevant pages.

Action Step 3:

- **Tool:** Google Search
- **Sub-goal:** Perform a Google search to find more detailed biographical information about Olivera Despina and her family members.
- **Command:**
`tool.execute(query="Who is the mother-in-law of Olivera Despina")`
- **Result:** Olivera Despina's mother-in-law was Gülçiçek Hatun. Olivera Despina was a Serbian princess who married Ottoman Sultan Bayezid I after the Battle of Kosovo in 1389. Gülçiçek Hatun was the first wife of Sultan Murad I and the mother of Bayezid I.

Execution Verifier: PASS, STOP

Solution Generator: Multi-tool verification approach successfully identified **Gülçiçek Hatun** as Olivera Despina's mother-in-law through systematic research.

Failure Case: AGENTFLOW w/o Flow-GRPO

Action Step 1:

- **Tool:** Google Search
- **Sub-goal:** Search for information about the film "Pugachev" (1978) and identify the director and any awards they received.
- **Command:**
`tool.execute(query="What is the award that the director of film Pugachev (1978 Film) received?", add_citations=True)`
- **Result:** Alexey Saltykov, the director of the 1978 film "Pugachev," received the title of People's Artist of the RSFSR in 1980 and the Order of the Badge of Honour for his contributions to cinema. While the film "Pugachev" itself was honored with a special prize at the All-Union Film Festival in 1979, this award was given to the film rather than specifically to Saltykov for his direction of it.
- **Note:** This step shows the agent working on a completely different question than asked.

Action Step 2:

- **Tool:** Wikipedia Search
- **Sub-goal:** Retrieve information about the personal awards received by Alexey Saltykov for directing the film "Pugachev" (1978).
- **Command:**
`tool.execute(query="Alexey Saltykov awards for Pugachev 1978")`
- **Result:** Returned pages about "Alexey Saltykov (director)", "Catherine the Great", and "Imperial Russian Army" but no relevant information about the original question.

Execution Verifier: STOP - Agent worked on wrong question, no useful information gathered for original query

Solution Generator: Tool attempts failed to address the actual question about Olivera Despina's mother-in-law. The agent became confused and worked on an unrelated question about the Pugachev film director.